南京郵電大學
Nanjing University of Posts and Telecommunications

Data Structures

Lecture 7: Sorted Sequences

Prof. Wang Haiyan
School of Computer Science
wanghy@njupt.edu.cn

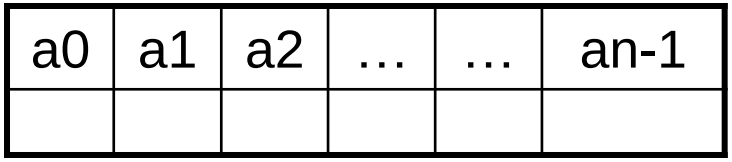
Reminder: Linear structures

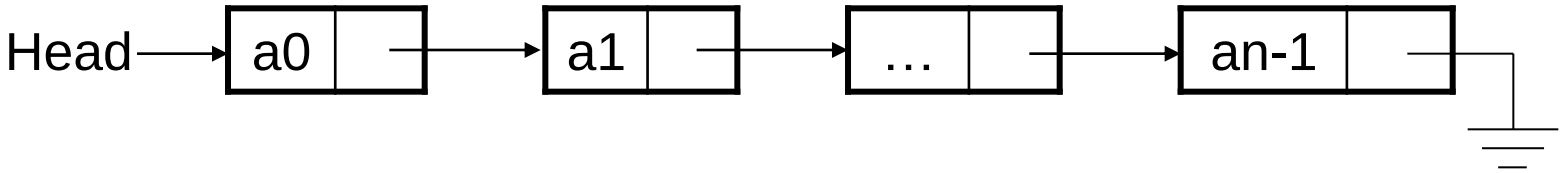
In lecture 4 we started to look at linear data structures and the Dictionary ADT

We know the following:

1. Retrieving an element from an array given its index is O(1)

2. Retrieving an element from a linked list given its index is O(n)





Reminder: Linear structures

In lecture 4 we started to look at linear data structures and the Dictionary ADT

We know the following:

1. Retrieving an element from an **array** given its **index** is O(1)
2. Retrieving an element from a **linked list** given its **index** is O(n)

However, if we are using an array or linked list to implement a dictionary, retrieval is done by **key**, not **index**.

1. This makes no difference to retrieval from a linked list, we still have to perform a linear search through the list, comparing keys.
2. Retrieval from an array-based implementation however, becomes O(n) as we can no longer calculate an address directly.

If the dictionary is stored in sorted form, matters change slightly…

Reminder: Linear structures

We now know how to sort a sequence of elements by key. We can then use binary search to retrieve elements from our dictionary using binary search on an array:

- Retrieval becomes O(log n) – this is enormously more efficient than O(n)

However, for linked lists we cannot jump into the middle and so even if the list is sorted we must still perform a linear search: O(n)

Maintaining a dictionary in sorted form introduces an overhead for insertions:

- For both linked lists and arrays we can no longer just add the element to the front/end – we must find the correct position and insert (involves shuffling elements for an array)

Reminder: Linear structures

So using linear structures, these are our options for implementing a dictionary:

Dictionary operation	Unsorted array	Sorted array	Unsorted list	Sorted list
`Lookup(D,k)`	O(n)	O(log n)	O(n)	O(n)
`Insert(D,v)`	O(1)	O(n)	O(1)	O(n)
`Delete(D,k)`	O(n)	O(n)	O(n)	O(n)
`IsPresent(D,k)`	O(n)	O(log n)	O(n)	O(n)

Additional considerations are that lists can make use of any available free memory while arrays require a contiguous chunk and that lists are more flexible for efficient insertion of subsequences etc.

Reminder: Linear structures

So using linear structures, these are our options for implementing a dictionary:

Dictionary operation	Unsorted array	Sorted array	Unsorted list	Sorted list
`Lookup(D,k)`	O(n)	O(log n)	O(n)	O(n)
`Insert(D,v)`	O(1)	O(n)	O(1)	O(n)
`Delete(D,k)`	O(n)	O(n)	O(n)	O(n)
`IsPresent(D,k)`	O(n)	O(log n)	O(n)	O(n)

Additional considerations are that lists can make use of any available free memory while arrays require a contiguous chunk and that lists are more flexible for efficient insertion of subsequences etc.

Conclusion: none of these will suffice for a practical implementation of a dictionary containing a large number of elements

If we move beyond linear structures, we can implement a dictionary in a way that efficiently supports these operations

Introduction to Trees

A ***tree*** is an abstract data type, comprising ***nodes*** joined by ***branches***

Nodes are defined by their:

- ***Degree***: how many branches leave the node
- ***Depth***: number of branches from the root to the node

The ***root*** is the only node with depth 0

Only one branch enters a node

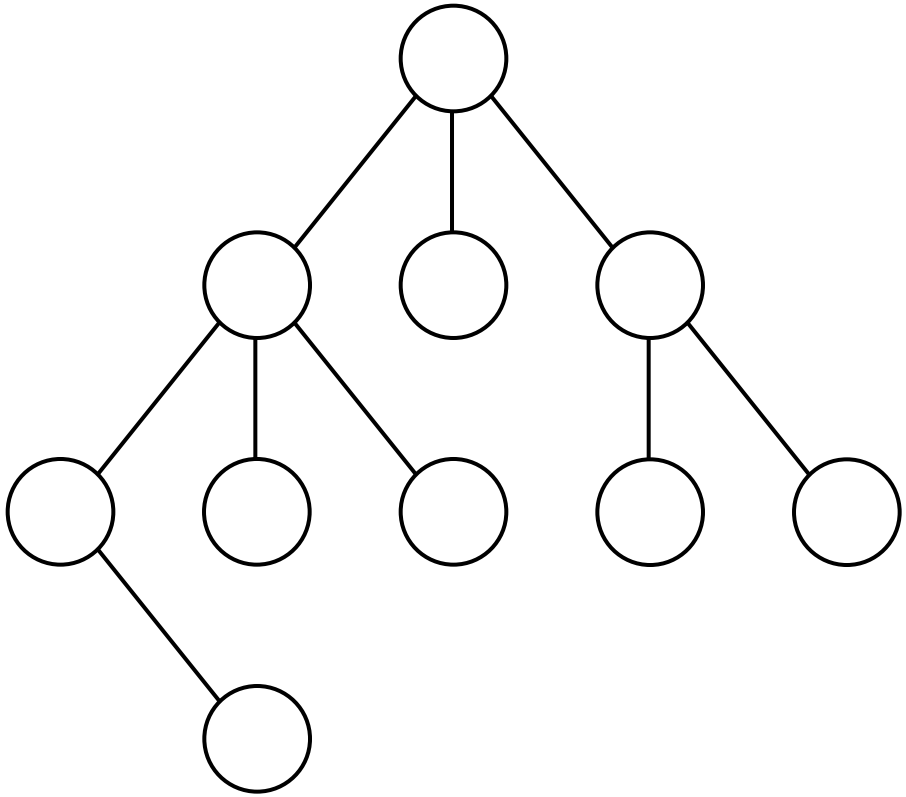
Nodes with degree 0 are called ***leaves***

A tree is a special case of a graph (think back to ICM/maths modules) in which any two vertices are connected by exactly one path

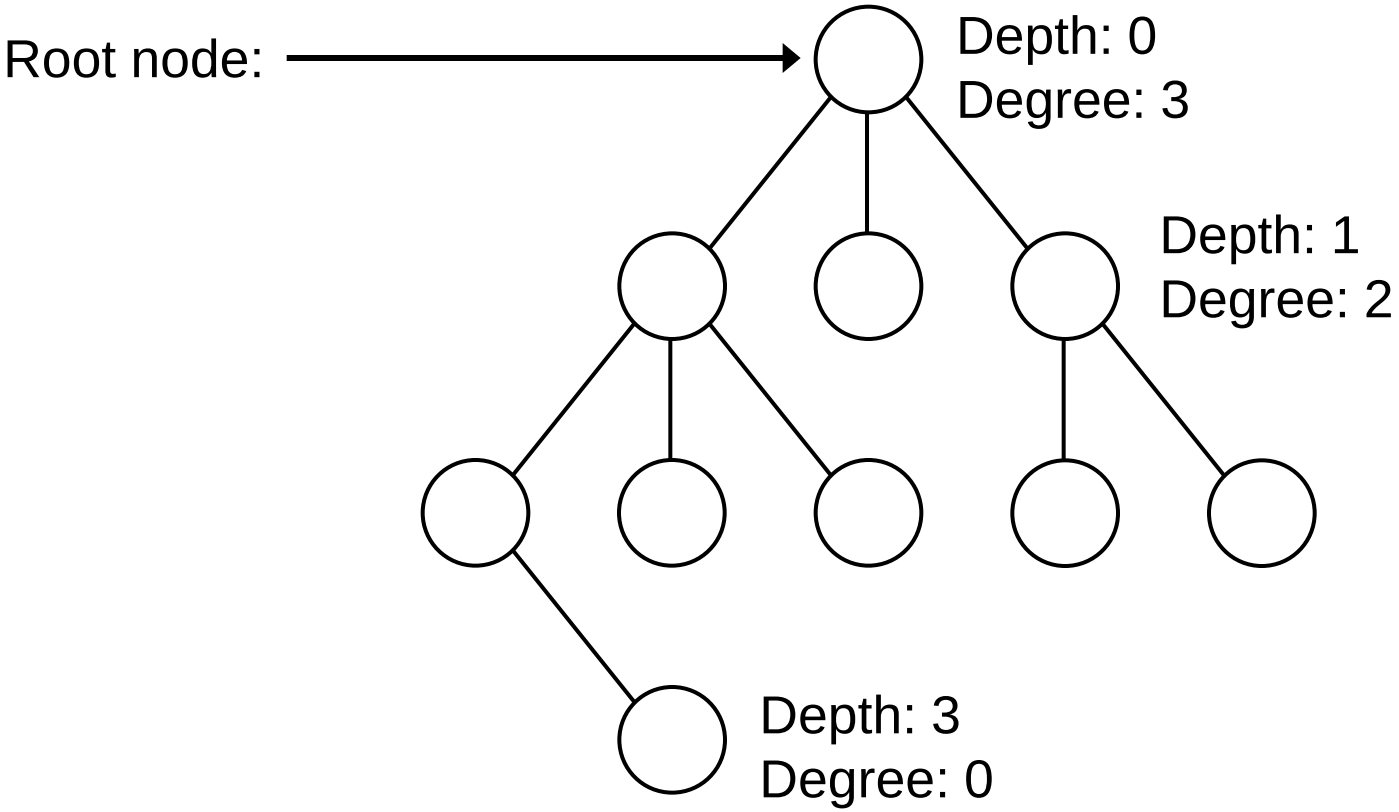
We will look at general graphs in some detail in the second half of this course

Introduction to Trees

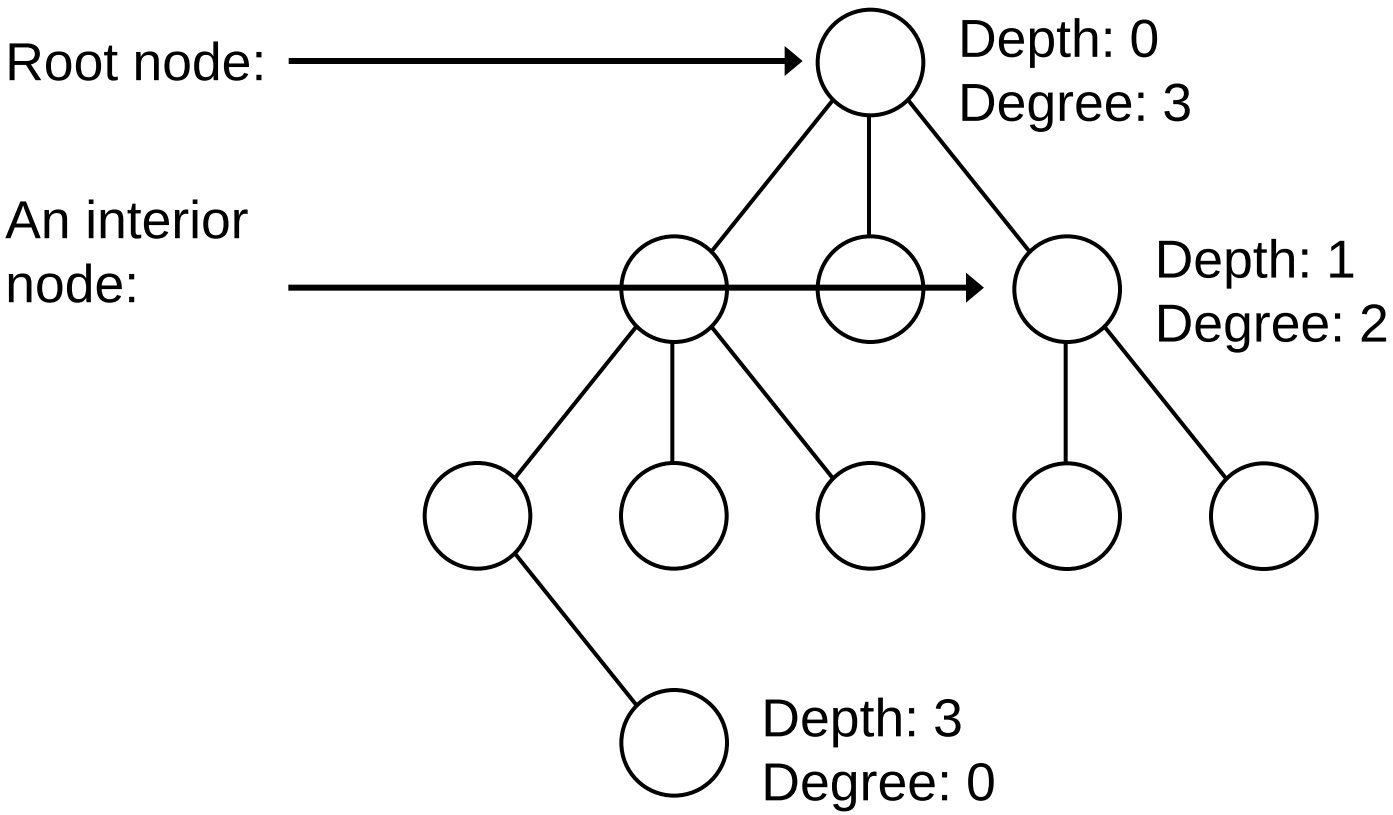
Root



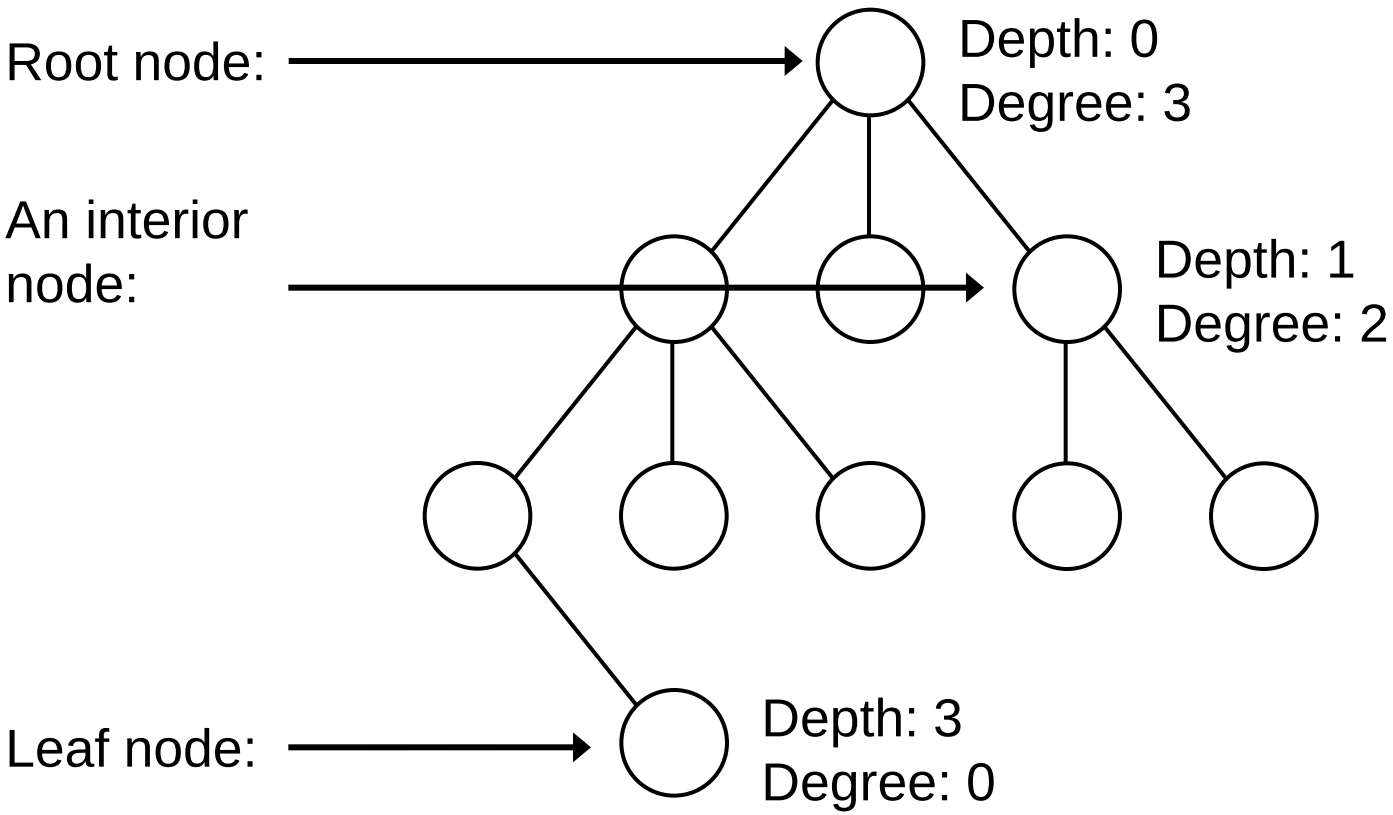
Introduction to Trees



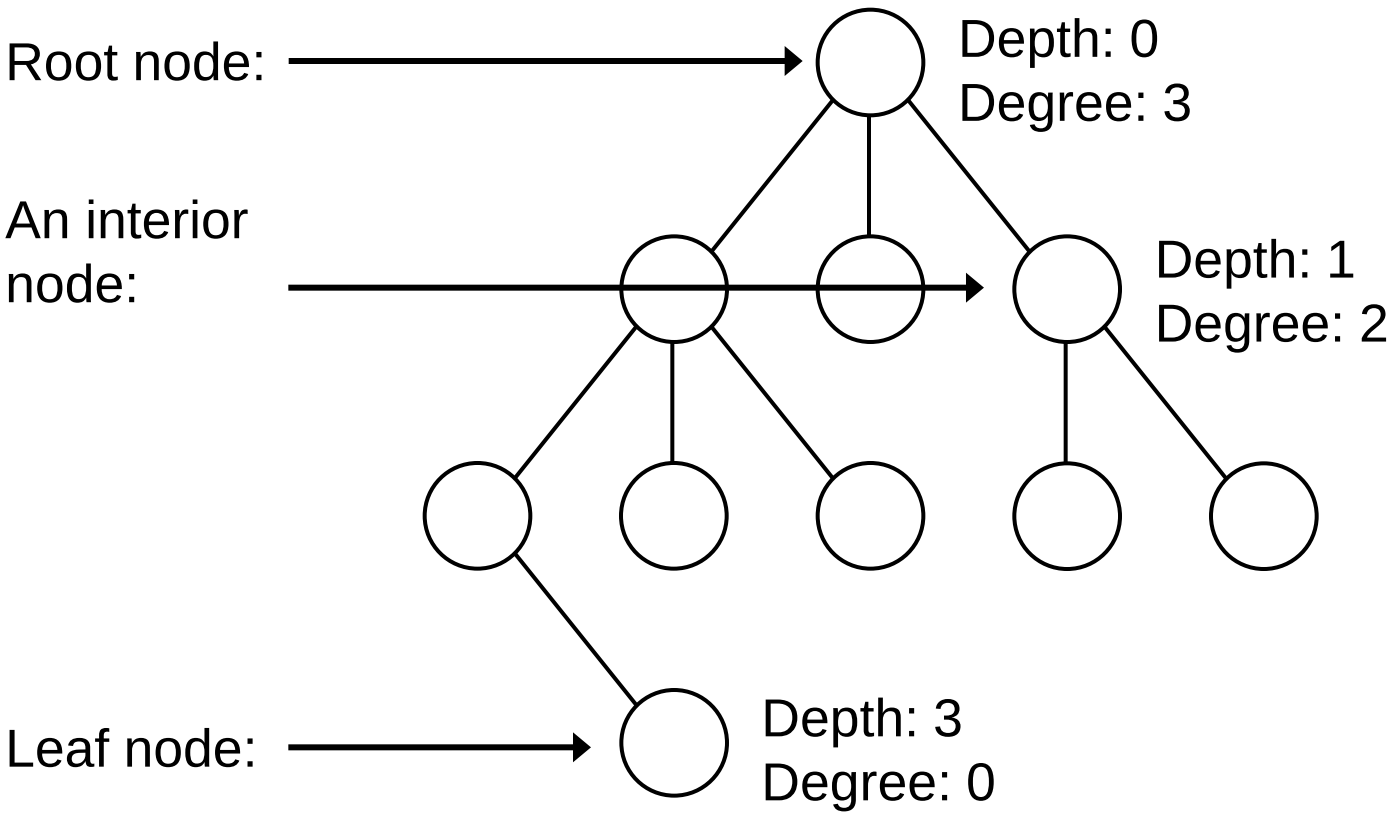
Introduction to Trees



Introduction to Trees

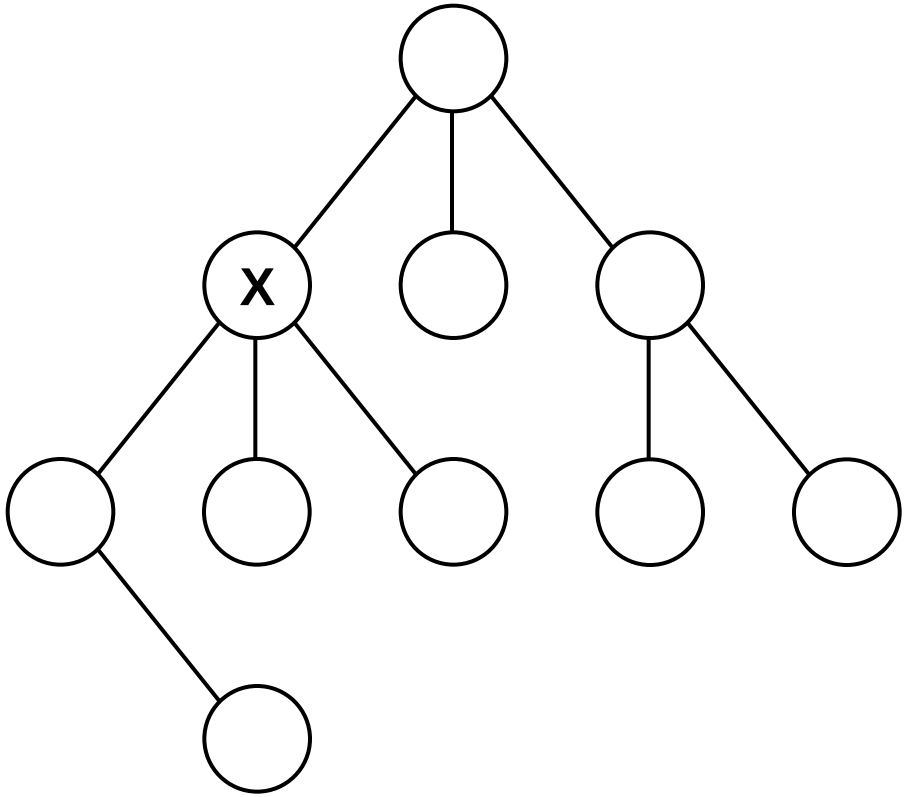


Introduction to Trees

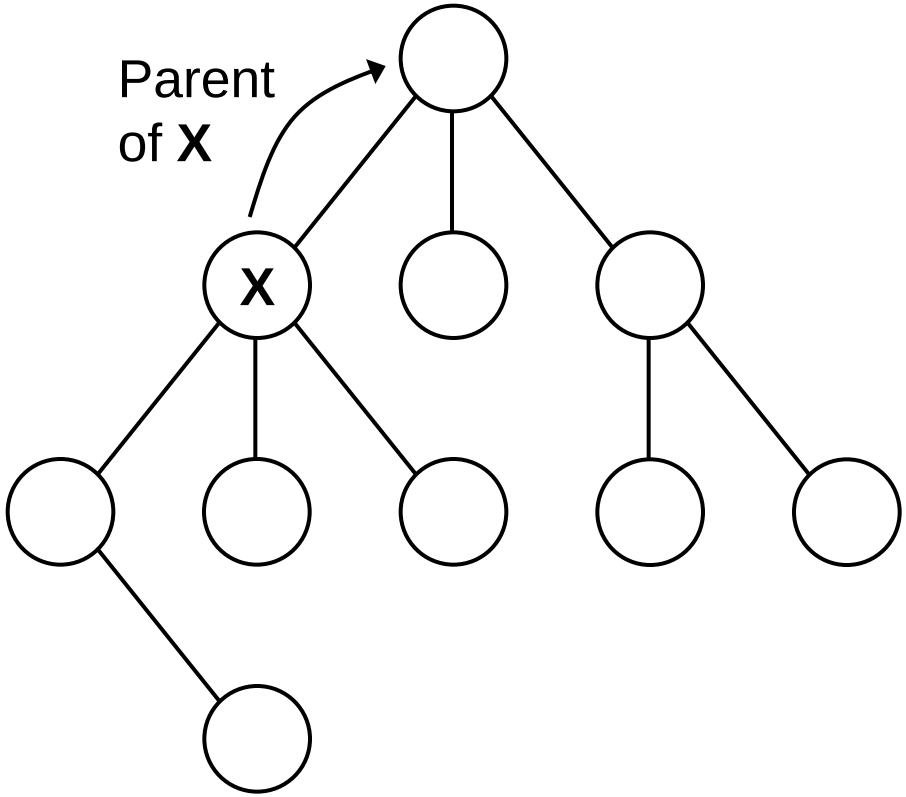


Trees may store data (records) in: 1. every node, 2. leaf nodes only or 3. nodes contain pointers into another structure which holds records (see Mehlhorn for an example using doubly linked lists)

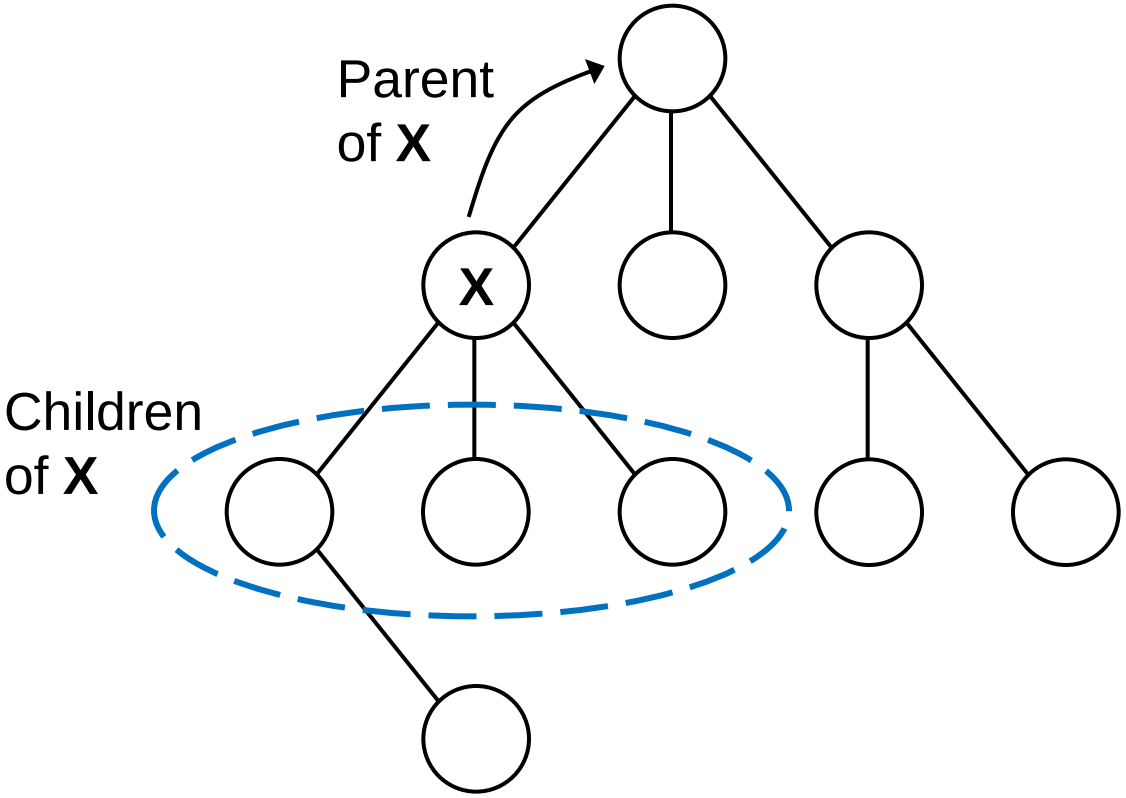
Introduction to Trees



Introduction to Trees



Introduction to Trees



Binary Search Trees

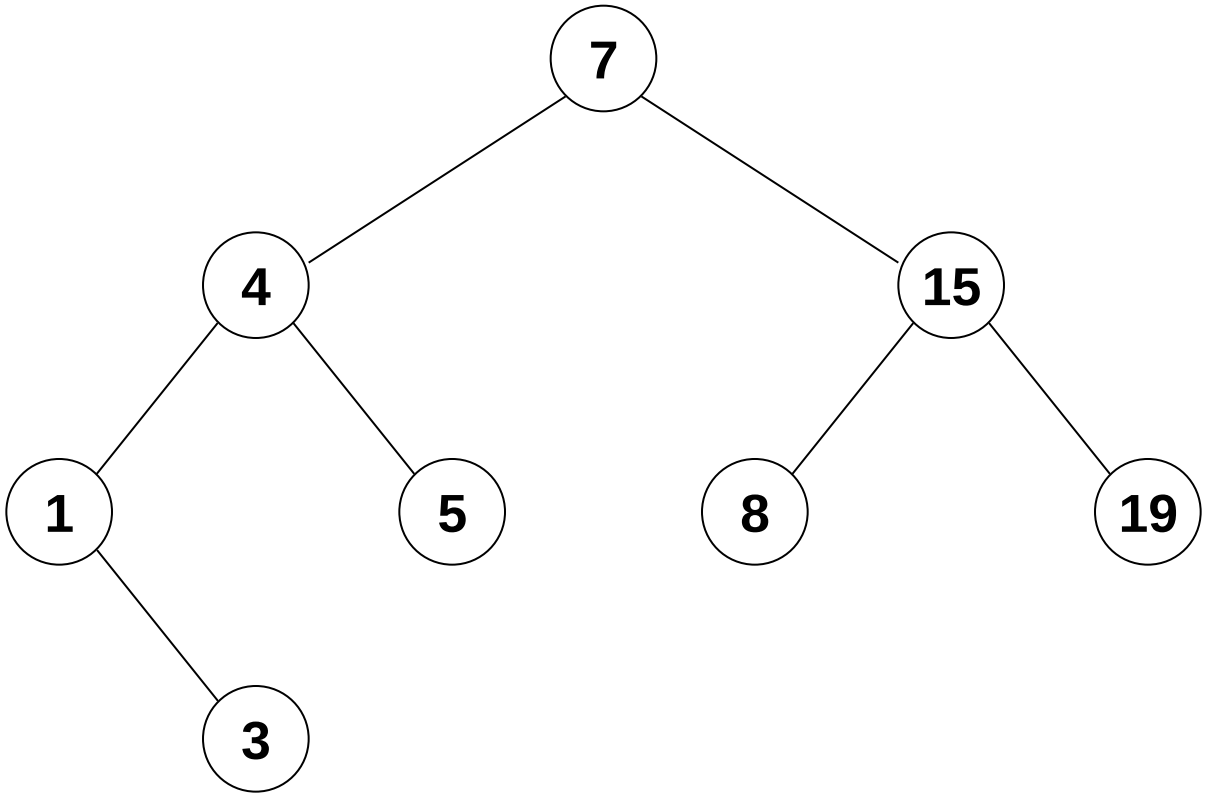
A ***binary tree*** is a special case of a tree in which nodes have a degree of at most 2

The two children of a node are called the left and right child

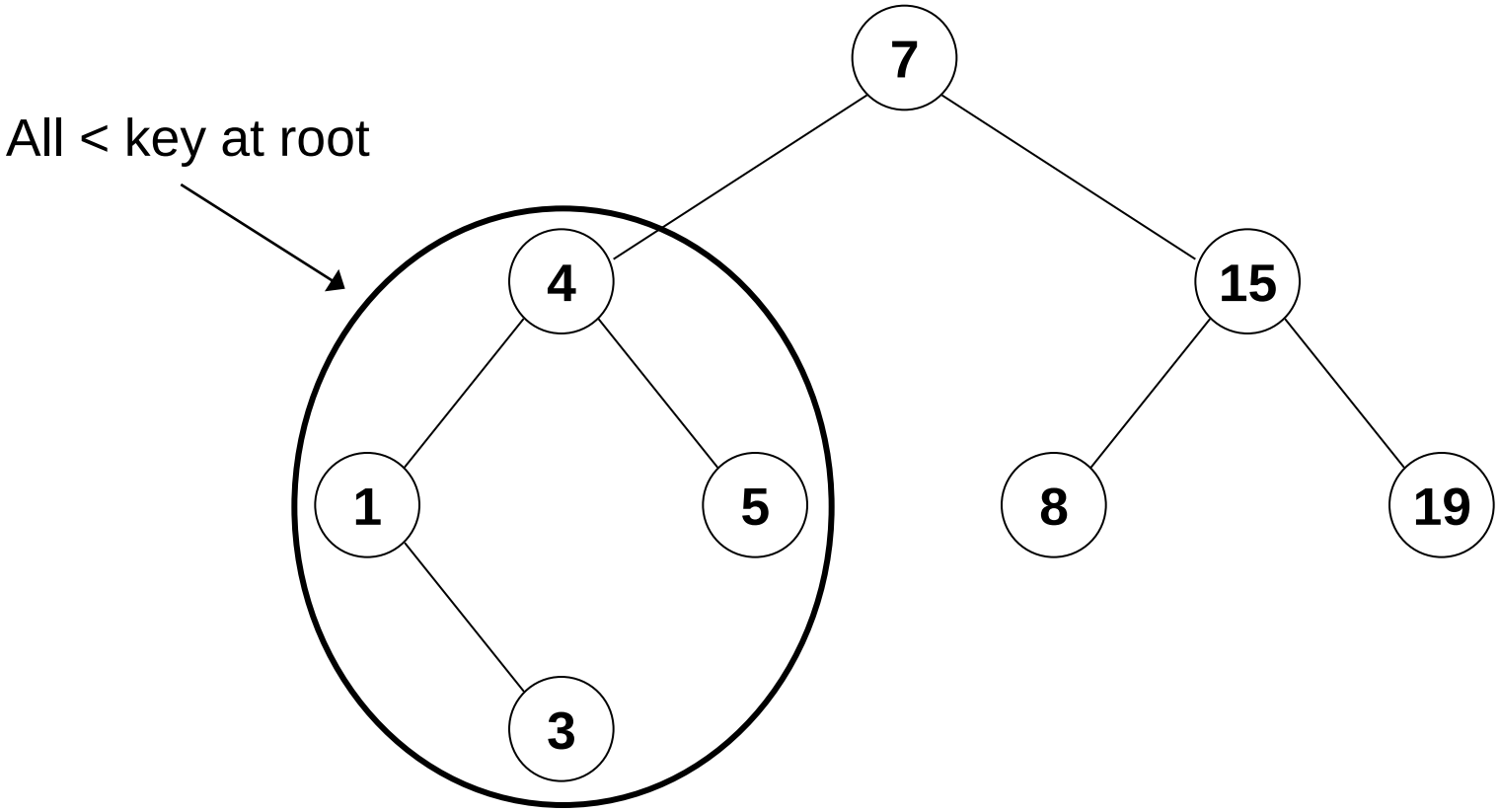
Binary search trees are a type of binary tree which are extremely useful for storing and retrieving ordered data:

- Each node is labelled with a key
- Nodes in the left subtree have keys smaller than that of the root
- Nodes in the right subtree have keys greater than that of the root

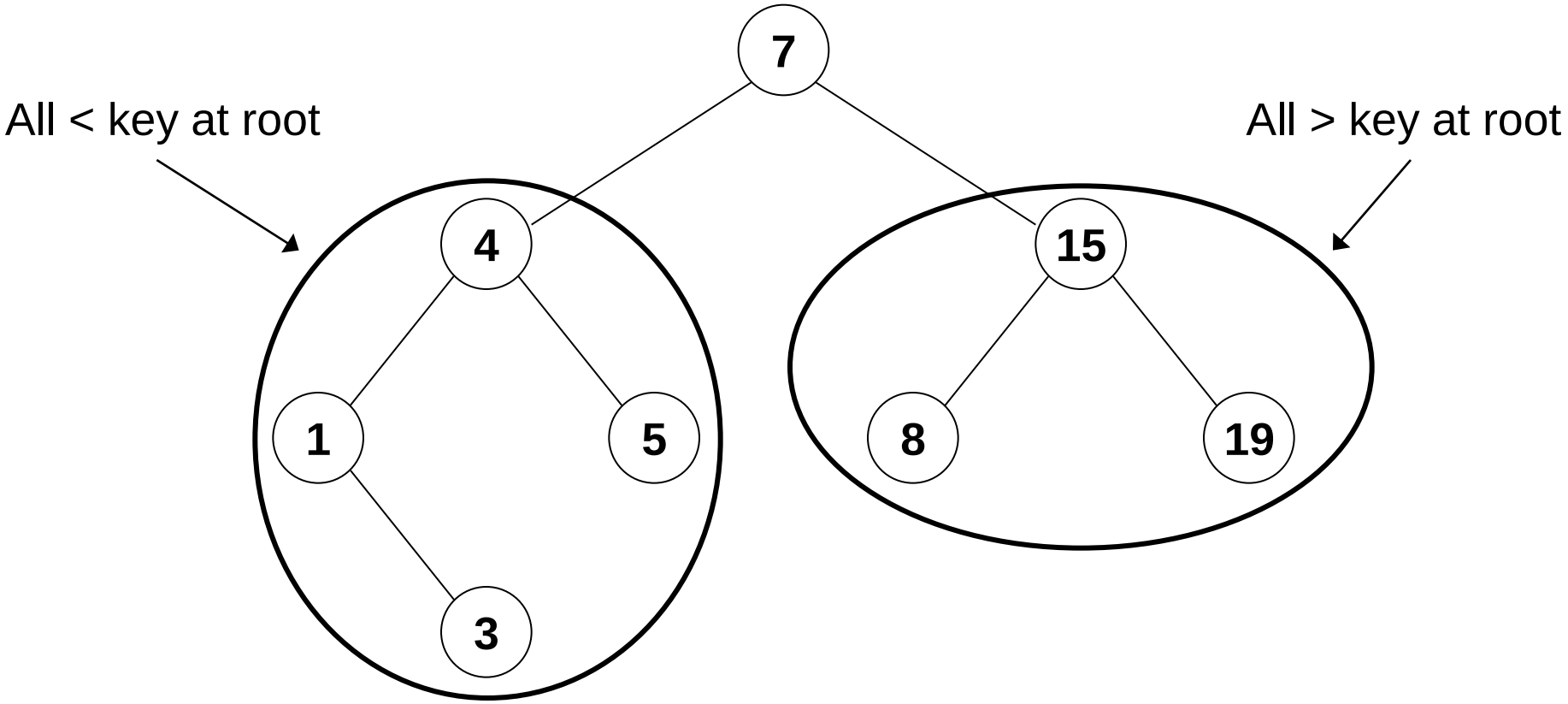
Binary Search Tree Example



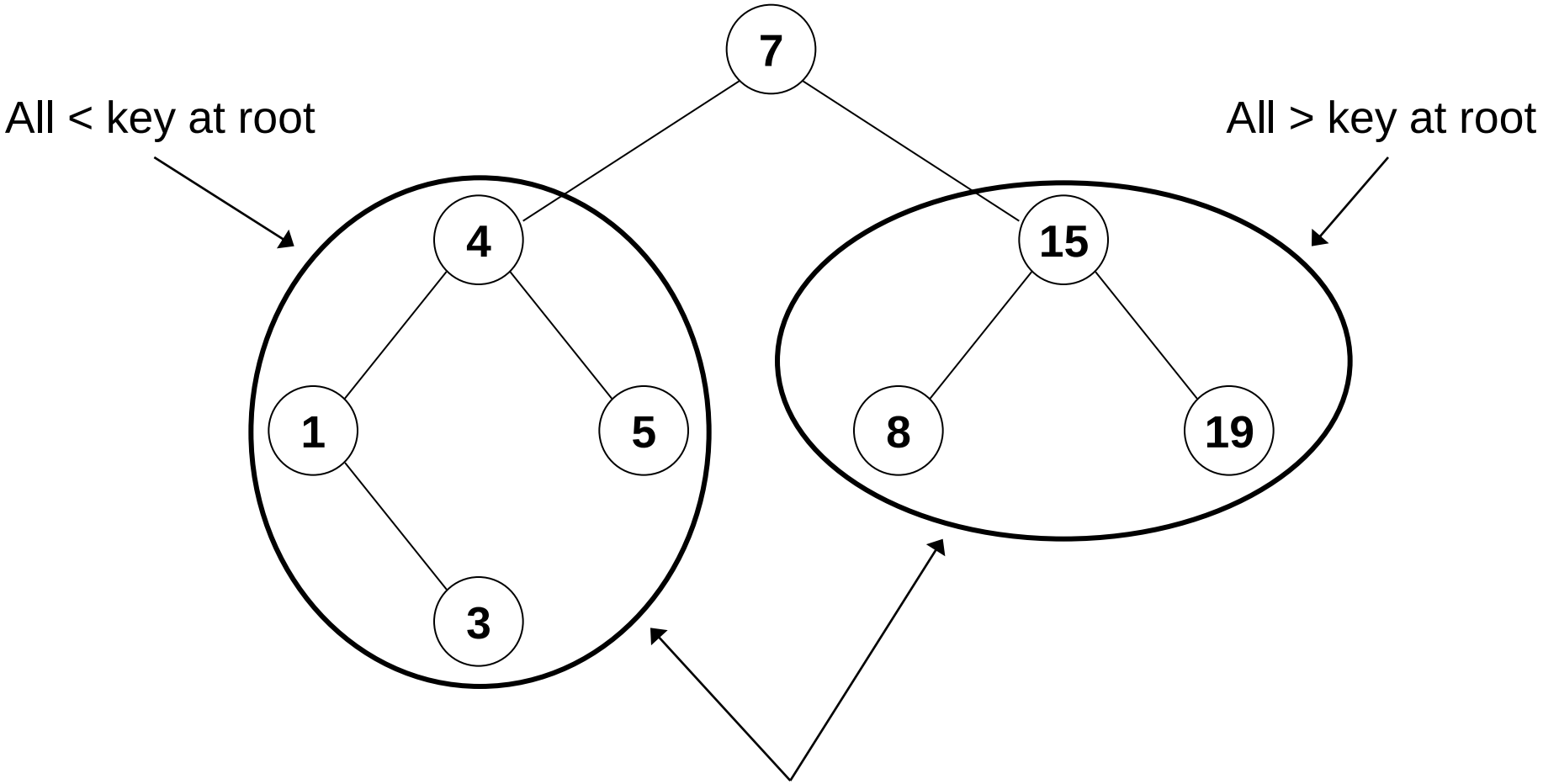
Binary Search Tree Example



Binary Search Tree Example



Binary Search Tree Example



Binary Search Trees

Recall: all our ADTs must ultimately be implemented using concrete data structures – linked or contiguous

Q: Can a binary tree be implemented using linked or contiguous concrete structures (or both)?

Binary Search Trees

Recall: all our ADTs must ultimately be implemented using concrete data structures – linked or contiguous

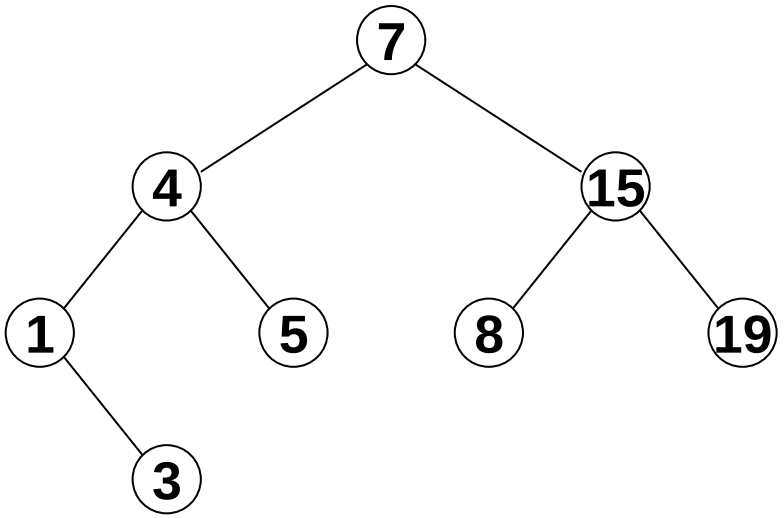
Q: Can a binary tree be implemented using linked or contiguous concrete structures (or both)?

A: Yes, both - but linked structures are much more common because they are more flexible and waste less space

(Note that in practice, records will contain data in addition to the key – this is excluded from the following examples)

Binary Search Tree as an Array

To store a binary tree in an array:



We place the root node at index location 0, followed by its left, then right children, followed by the left and right children of the left node and the left and right children of the right node and so on:

Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Contents:	7	4	15	1	5	8	19	-1	3	-1	-1	-1	-1	-1	-1

Given the index, i, of a node, we can calculate the index of its parent and children:

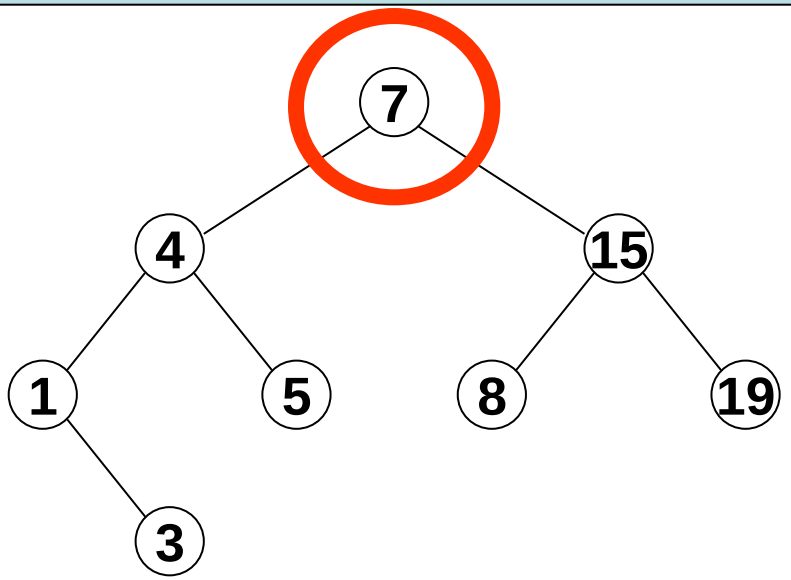
$$\text{Parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

$$\text{RightChild}(i) = 2i + 2$$

$$\text{LeftChild}(i) = 2i + 1$$

Binary Search Tree as an Array

To store a binary tree in an array:



We place the root node at index location 0, followed by its left, then right children, followed by the left and right children of the left node and the left and right children of the right node and so on:

Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Contents:	7	4	15	1	5	8	19	-1	3	-1	-1	-1	-1	-1	-1

Given the index, i, of a node, we can calculate the index of its parent and children:

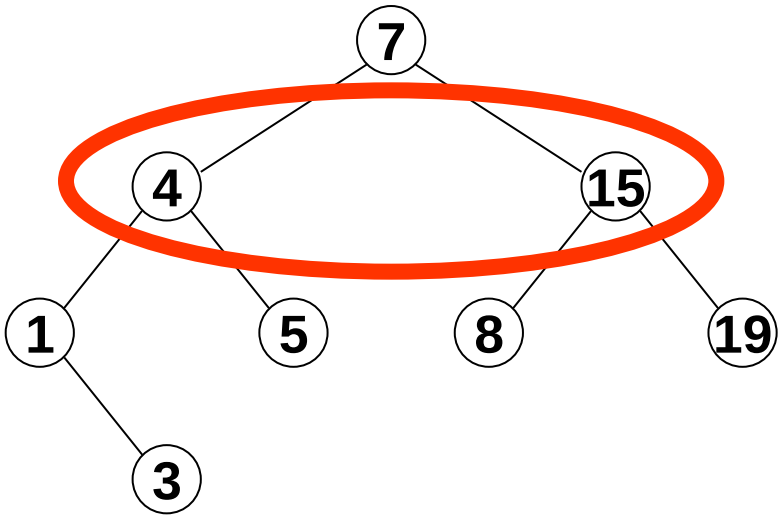
$$\text{Parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

$$\text{RightChild}(i) = 2i + 2$$

$$\text{LeftChild}(i) = 2i + 1$$

Binary Search Tree as an Array

To store a binary tree in an array:



We place the root node at index location 0, followed by its left, then right children, followed by the left and right children of the left node and the left and right children of the right node and so on:

Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Contents:	7	4	15	1	5	8	19	-1	3	-1	-1	-1	-1	-1	-1

Given the index, i, of a node, we can calculate the index of its parent and children:

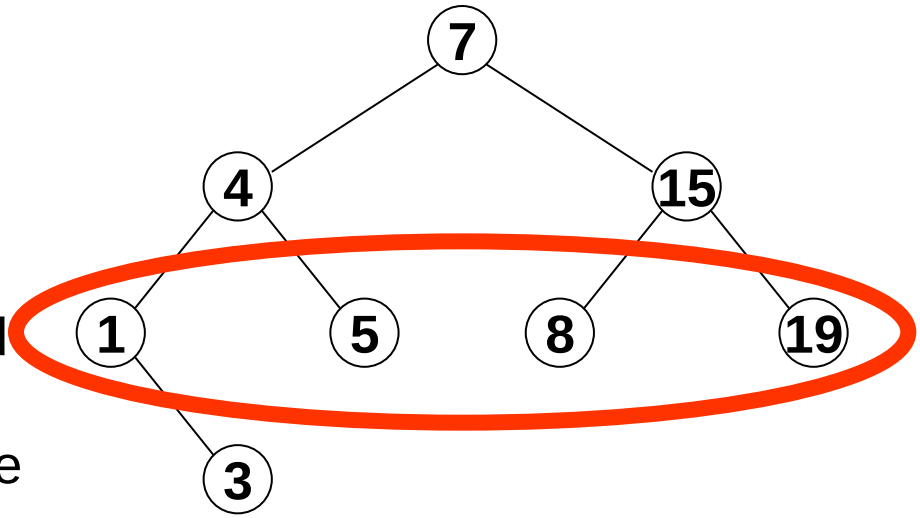
$$\text{Parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

$$\text{RightChild}(i) = 2i + 2$$

$$\text{LeftChild}(i) = 2i + 1$$

Binary Search Tree as an Array

To store a binary tree in an array:



We place the root node at index location 0, followed by its left, then right children, followed by the left and right children of the left node and the left and right children of the right node and so on:

Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Contents:	7	4	15	1	5	8	19	-1	3	-1	-1	-1	-1	-1	-1

Given the index, i, of a node, we can calculate the index of its parent and children:

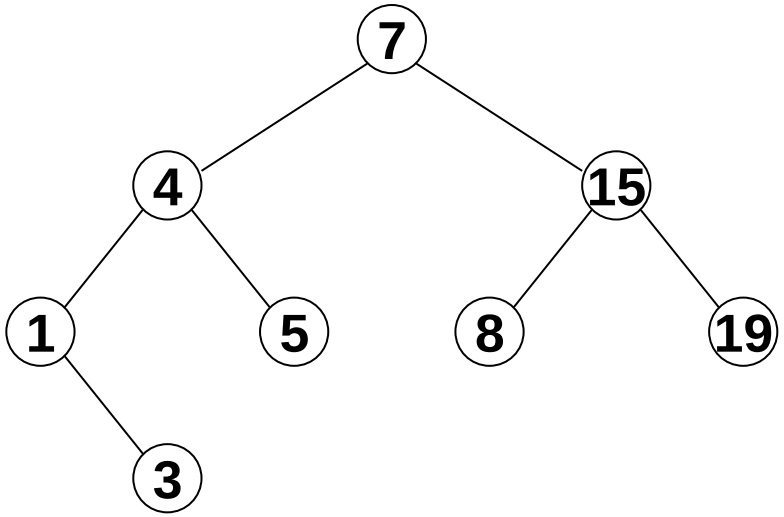
$$\text{Parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

$$\text{RightChild}(i) = 2i + 2$$

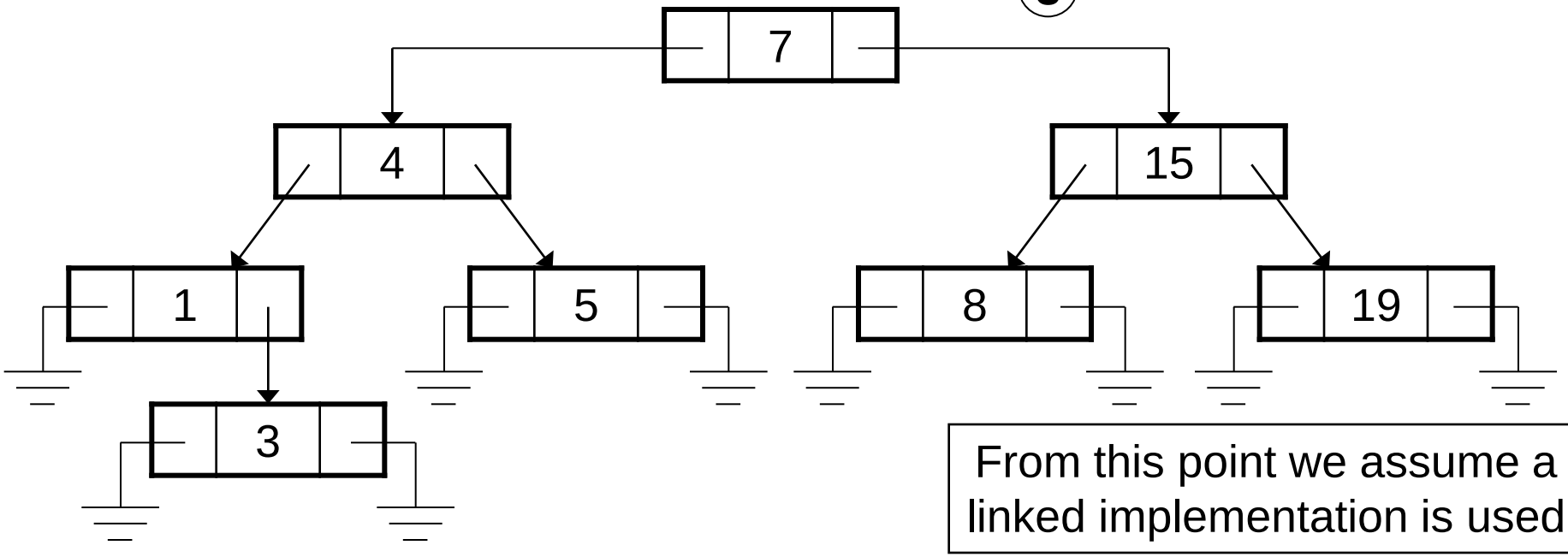
$$\text{LeftChild}(i) = 2i + 1$$

Binary Search Tree as a Linked Structure

To store a binary tree in a linked structure:



Each node consists of a record containing the key/datum plus pointers to records representing its right and left children:



From this point we assume a linked implementation is used

Retrieving an Element from a Binary Search Tree

- Binary search trees are a recursive structure
- Implies they will be handled using recursive algorithms
- Retrieving data from a binary search tree is a recursive process:

Retrieving an Element from a Binary Search Tree

- Binary search trees are a recursive structure
- Implies they will be handled using recursive algorithms
- Retrieving data from a binary search tree is a recursive process:

Function $BinarySearch(N : \textbf{Pointer to } \text{Element}; K : \text{Key})$

```
1: if N = NULL then
2:     return NULL
3: else if K = N → key then
4:     return N
5: else if K < N → key then
6:     return BinarySearch(N → left, K)
7: else
8:     return BinarySearch(N → right, K)
9: end if
```

(If you don't follow pseudocode notation for pointers, see Mehlhorn pp. 26-27)

Retrieving an Element from a Binary Search Tree

- Binary search trees are a recursive structure
- Implies they will be handled using recursive algorithms
- Retrieving data from a binary search tree is a recursive process:

Function *BinarySearch*(N : **Pointer to** Element; K : Key)

```
1: if N = NULL then
2:     return NULL
3: else if K = N → key then
4:     return N
5: else if K < N → key then
6:     return BinarySearch(N → left, K)
7: else
8:     return BinarySearch(N → right, K)
9: end if
```

Null pointer reached – key not in tree

(If you don't follow pseudocode notation for pointers, see Mehlhorn pp. 26-27)

Retrieving an Element from a Binary Search Tree

- Binary search trees are a recursive structure
- Implies they will be handled using recursive algorithms
- Retrieving data from a binary search tree is a recursive process:

Function $BinarySearch(N : \textbf{Pointer to } \text{Element}; K : \text{Key})$

```
1: if N = NULL then
2:     return NULL
3: else if K = N → key then
4:     return N
5: else if K < N → key then
6:     return BinarySearch(N → left, K)
7: else
8:     return BinarySearch(N → right, K)
9: end if
```

Null pointer reached – key not in tree

Required key found, return this node

(If you don't follow pseudocode notation for pointers, see Mehlhorn pp. 26-27)

Retrieving an Element from a Binary Search Tree

- Binary search trees are a recursive structure
- Implies they will be handled using recursive algorithms
- Retrieving data from a binary search tree is a recursive process:

Function $BinarySearch(N : \textbf{Pointer to } \text{Element}; K : \text{Key})$

```
1: if N = NULL then
2:     return NULL
3: else if K = N → key then
4:     return N
5: else if K < N → key then
6:     return BinarySearch(N → left, K)
7: else
8:     return BinarySearch(N → right, K)
9: end if
```

Null pointer reached – key not in tree

Required key found, return this node

Otherwise, recursively search either left or right subtree depending on whether key of current node is less/ greater than desired

(If you don't follow pseudocode notation for pointers, see Mehlhorn pp. 26-27)

Retrieving an Element from a Binary Search Tree

Q: What is the complexity of retrieving an element from a binary search tree?

Retrieving an Element from a Binary Search Tree

Q: What is the complexity of retrieving an element from a binary search tree?

A: Best case = O(1) – root is the node we're looking for

Worst case for a tree of depth h = O(h) – we have to follow branches right down to the deepest leaf

In-order Traversal of a Binary Search Tree

An ***in-order traversal*** visits every node in a binary search tree and processes them in order

Given a node, process left subtree, then node, then right subtree, e.g.:

Function $PrintInOrder(N : \textbf{Pointer to } \text{Element})$
1: **if** $N \not\doteq$ `NULL` **then**
2: $\quad PrintInOrder(N \rightarrow \text{left})$
3: $\quad print(N \rightarrow \text{datum})$
4: $\quad PrintInOrder(N \rightarrow \text{right})$
5: **end if**

In-order Traversal of a Binary Search Tree

An ***in-order traversal*** visits every node in a binary search tree and processes them in order

Given a node, process left subtree, then node, then right subtree, e.g.:

Function $PrintInOrder(N : \textbf{Pointer to } \text{Element})$
1: **if** $N \not= \texttt{NULL}$ **then**
2: $PrintInOrder(N \rightarrow \text{left})$
3: $print(N \rightarrow \text{datum})$
4: $PrintInOrder(N \rightarrow \text{right})$
5: **end if**

Q: Complexity of an in-order traversal?

In-order Traversal of a Binary Search Tree

An ***in-order traversal*** visits every node in a binary search tree and processes them in order

Given a node, process left subtree, then node, then right subtree, e.g.:

Function $PrintInOrder(N : \textbf{Pointer to } \text{Element})$
1: **if** $N \not= \texttt{NULL}$ **then**
2: $PrintInOrder(N \rightarrow \text{left})$
3: $print(N \rightarrow \text{datum})$
4: $PrintInOrder(N \rightarrow \text{right})$
5: **end if**

Q: Complexity of an in-order traversal?

A: O(n) – we do O(1) work for each of n nodes

Other Operations on Binary Search Trees

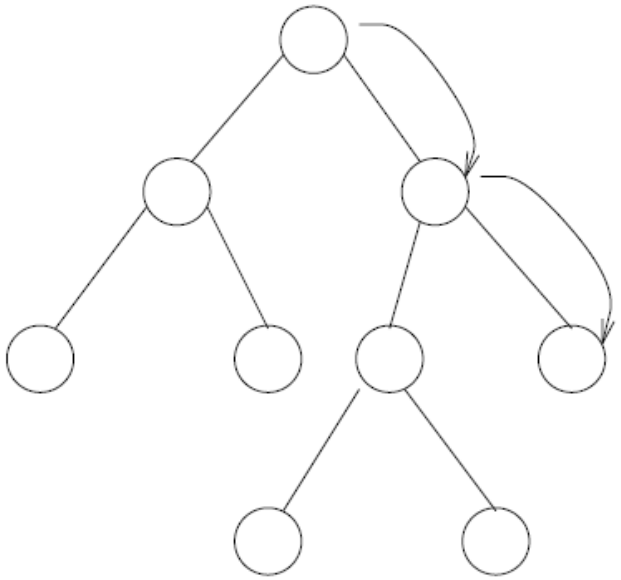
Q: How do we find the minimum and maximum elements in a binary search tree? What is the complexity?

Other Operations on Binary Search Trees

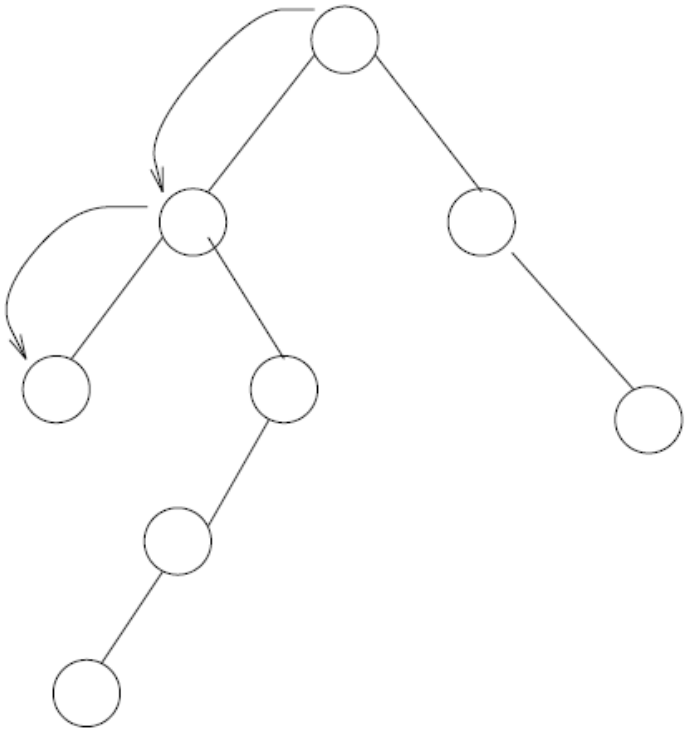
Q: How do we find the minimum and maximum elements in a binary search tree? What is the complexity?

A: Follow right/left to leaf. O(h) in both cases, e.g.:

Maximum



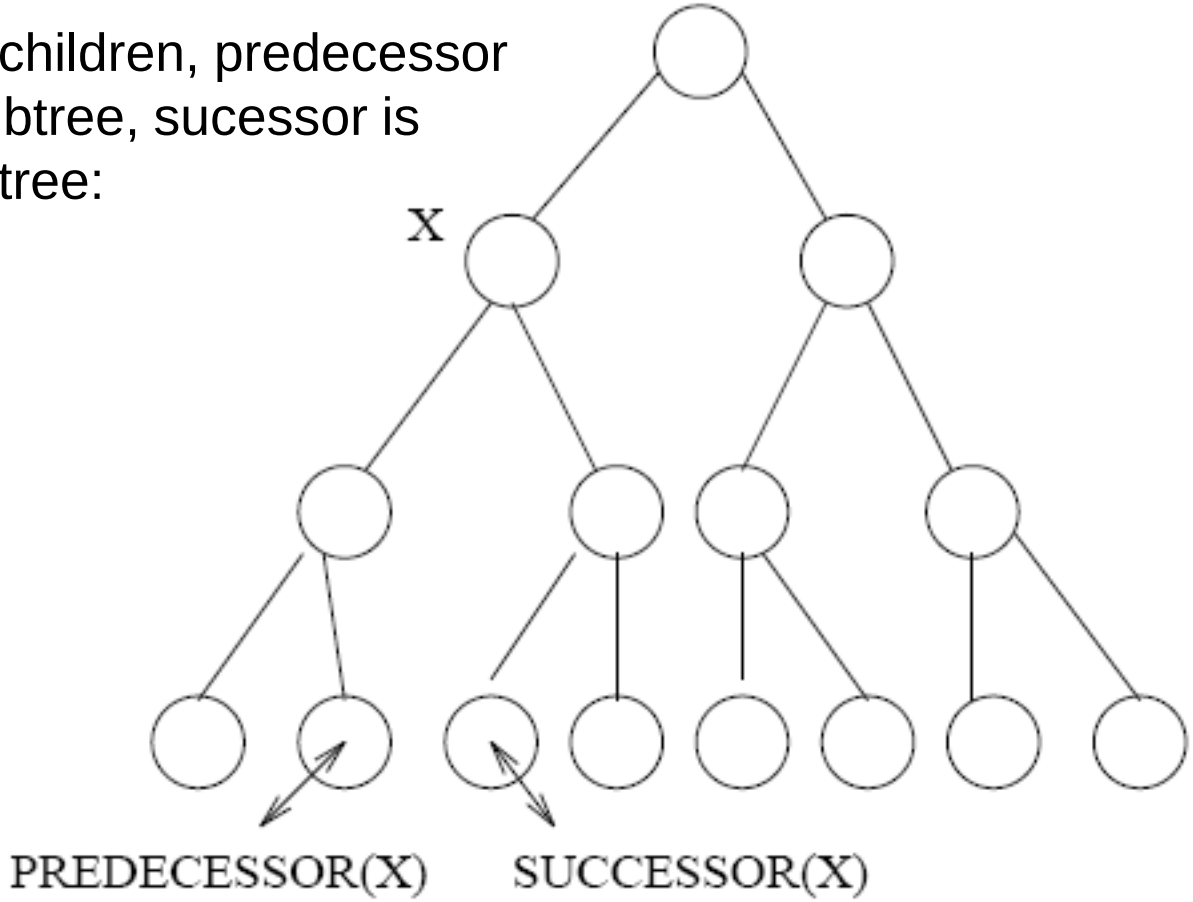
Minimum



Other Operations on Binary Search Trees

Other operations that we might wish to carry out on a binary tree: predecessor and successor

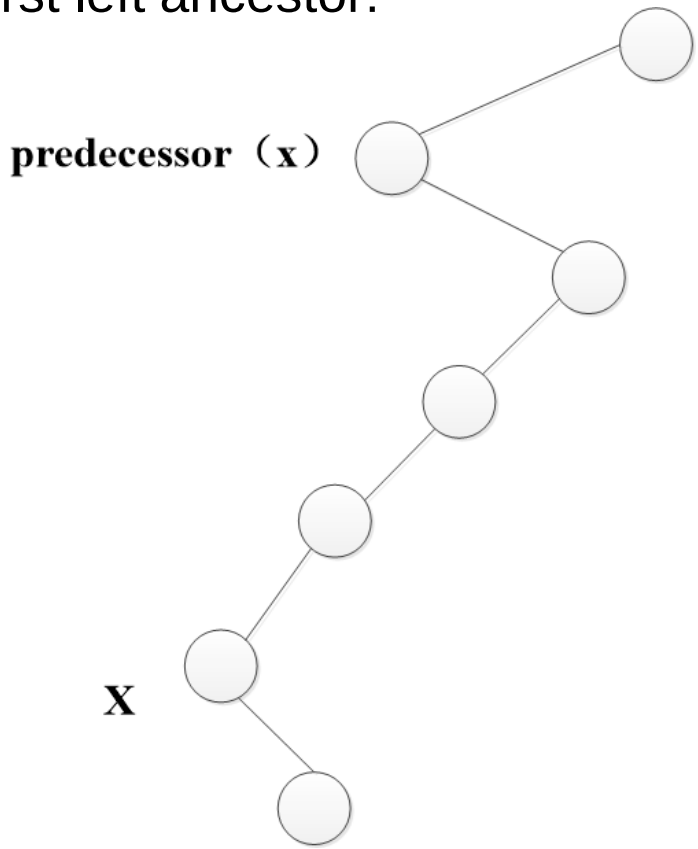
For nodes with children, predecessor is max of left subtree, sucessor is min of right subtree:



Other Operations on Binary Search Trees

Other operations that we might wish to carry out on a binary tree: predecessor and successor

For nodes with no left child, predecessor is first left ancestor:



Similarly, for nodes with no right child, successor is first right ancestor

Insertion into Binary Search Trees

Q: How do we insert something into a binary search tree, ensuring that it remains a binary search tree? What is the complexity?

Insertion into Binary Search Trees

Q: How do we insert something into a binary search tree, ensuring that it remains a binary search tree? What is the complexity?

A: Perform a binary search to find where it should go and set NULL pointer to point to new record

Setting the pointer and creating the record is only O(1) and as we've seen, the binary search is O(h), so insertion is also O(h)

Deletion from Binary Search Trees

Q: How do we delete something from a binary search tree, ensuring that it remains a binary search tree? What is the complexity?

Deletion from Binary Search Trees

Q: How do we delete something from a binary search tree, ensuring that it remains a binary search tree? What is the complexity?

A: First we have to find, O(h), then it depends on how many children the node has:

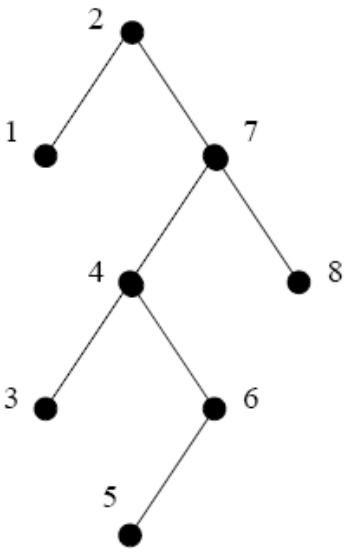
0 children (i.e. leaf) – Straightforward, just set pointer from parent to NULL and deallocate memory used for record of deleted node

1 child – Again straightforward, replace the deleted node with its single child

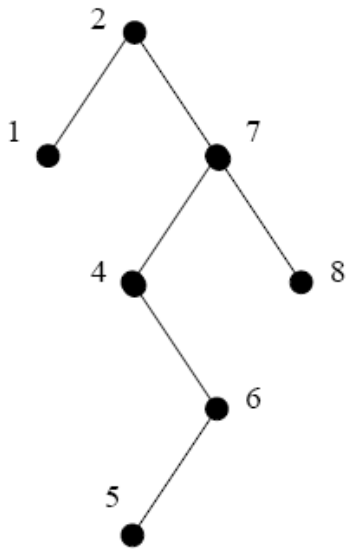
2 children – More complicated, replace the node with its successor and delete the successor

Deletion from Binary Search Trees

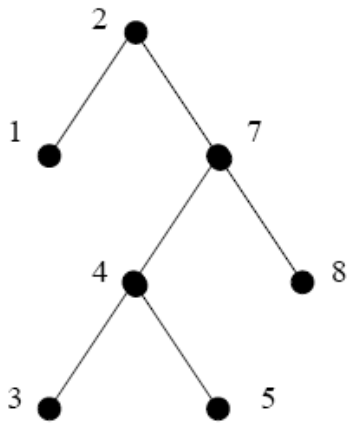
Initial tree



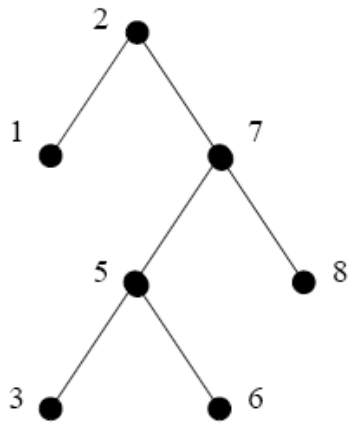
Delete node 3
(0 children)



Delete node 6
(1 child)



Delete node 4
(2 children)



- Note: the successor of a node with two children will always have at most one child (which will be a left child) – if it had a right child, it would not be the successor!
- So deleting the successor will be one of the straightforward cases
- Case 1 & 2 is O(1), case 3 is O(h) so deleting is O(h) including finding the node

Dictionaries as Binary Search Trees

From what we know so far, we could implement a dictionary as a binary search tree and every operation would have worst case complexity O(h):

Dictionary operation	Binary Search Tree
`Lookup(D,k)`	O(h)
`Insert(D,v)`	O(h)
`Delete(D,k)`	O(h)
`IsPresent(D,k)`	O(h)

So the performance of a binary search tree on these operations is determined entirely by its depth

Q: What determines the depth of our tree?

Dictionaries as Binary Search Trees

From what we know so far, we could implement a dictionary as a binary search tree and every operation would have worst case complexity O(h):

Dictionary operation	Binary Search Tree
`Lookup(D,k)`	O(h)
`Insert(D,v)`	O(h)
`Delete(D,k)`	O(h)
`IsPresent(D,k)`	O(h)

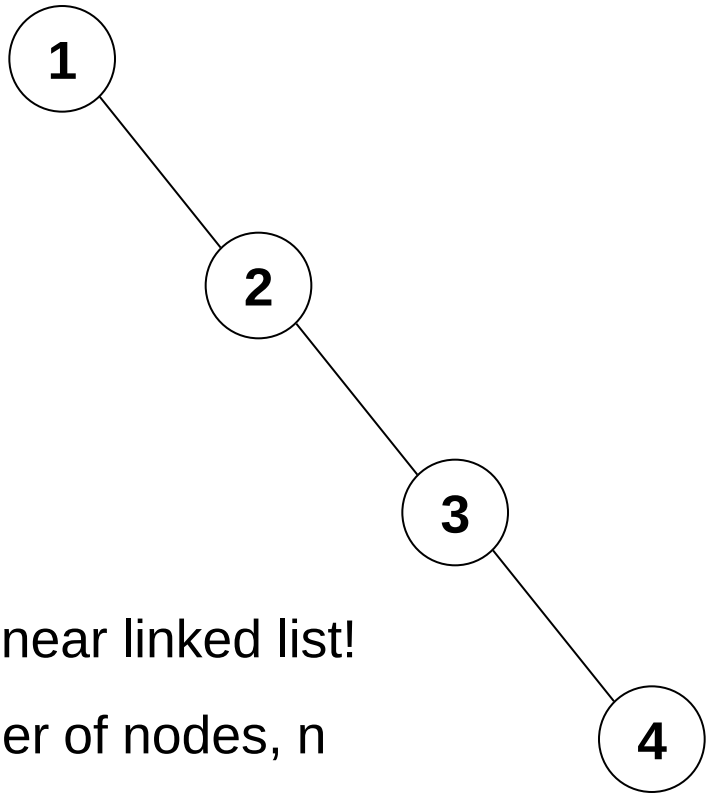
So the performance of a binary search tree on these operations is determined entirely by its depth

Q: What determines the depth of our tree?

A: The order of insertion

Binary Search Tree Depth

The worst case tree depth occurs when elements are inserted in order or reverse order, i.e.: insert(1), insert(2), insert(3), insert(4) …



We end up with a linear linked list!

Tree depth = number of nodes, n

All operations linear: O(n)

Binary Search Tree Depth

Q: What is the best case depth for a tree with n nodes?

Binary Search Tree Depth

Q: What is the best case depth for a tree with n nodes?

A: A tree in which as many nodes as possible have two children – this will maximise the number of nodes at each depth and hence minimise the maximum depth

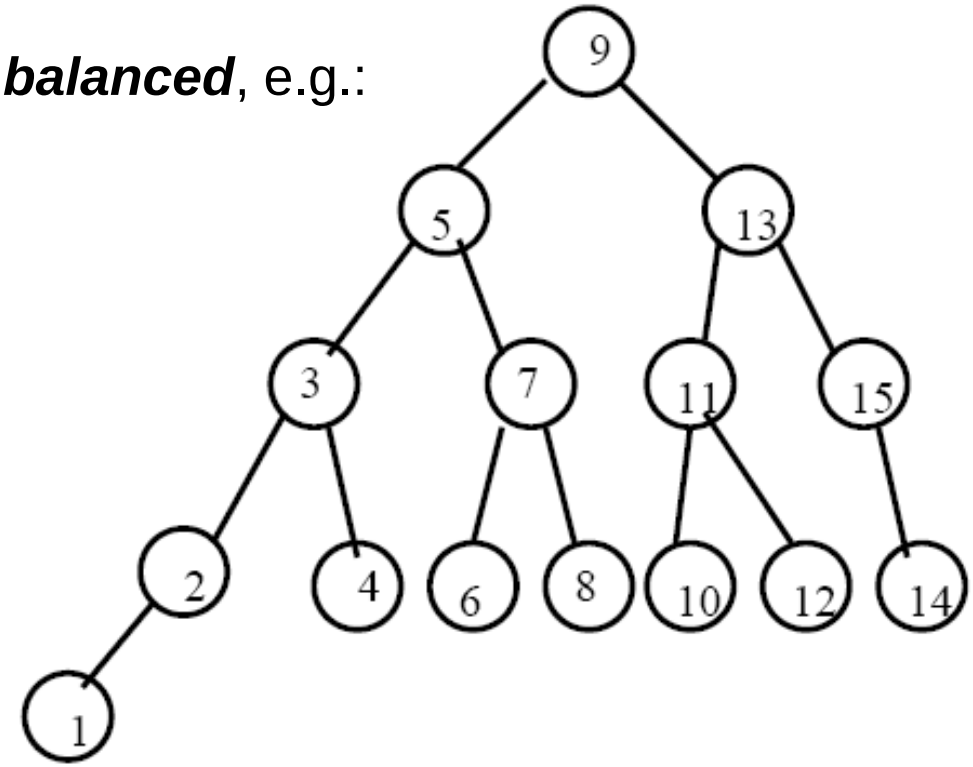
Such a tree is said to be ***perfectly balanced***, e.g.:

Binary Search Tree Depth

Q: What is the best case depth for a tree with n nodes?

A: A tree in which as many nodes as possible have two children – this will maximise the number of nodes at each depth and hence minimise the maximum depth

Such a tree is said to be ***perfectly balanced***, e.g.:

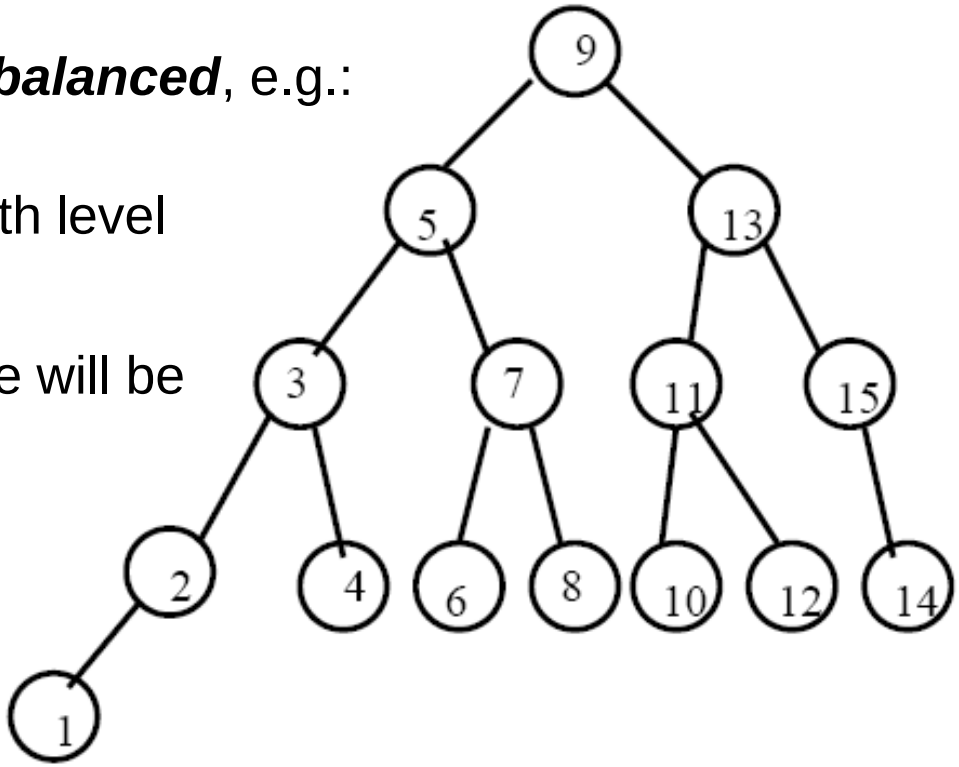


Binary Search Tree Depth

Q: What is the best case depth for a tree with n nodes?

A: A tree in which as many nodes as possible have two children – this will maximise the number of nodes at each depth and hence minimise the maximum depth

Such a tree is said to be ***perfectly balanced***, e.g.:



There are at most 2^k nodes at the kth level of a binary tree

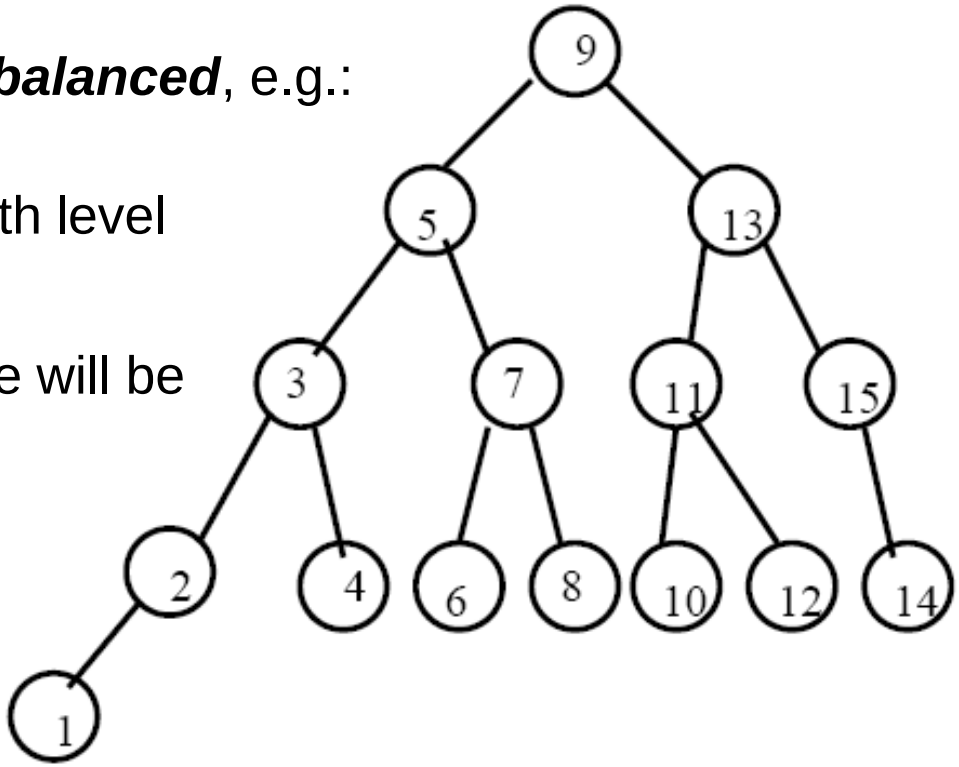
So in a perfectly balanced tree there will be 2^k nodes at all but the last level

Binary Search Tree Depth

Q: What is the best case depth for a tree with n nodes?

A: A tree in which as many nodes as possible have two children – this will maximise the number of nodes at each depth and hence minimise the maximum depth

Such a tree is said to be ***perfectly balanced***, e.g.:



There are at most 2^k nodes at the kth level of a binary tree

So in a perfectly balanced tree there will be 2^k nodes at all but the last level

Q: Hence, the depth of a perfectly balanced tree is?

Binary Search Tree Depth

Summing over nodes at each level gives total number of nodes so:

$$\sum_{i=1}^{h} 2^i \approx n$$

Solving for h gives:

$$h = \lceil \log_2 n \rceil$$

So in the best case (perfectly balanced tree) the tree depth is O(log n)

This means that all dictionary operations on a perfectly balanced BST would be O(log n) – this beats or matches all operations using a linear implementation

Conclusion: If we could somehow ensure our binary search tree was always perfectly balanced, we could execute all dictionary operations in O(log n) time

Binary Search Tree Depth

How about just hoping that the tree will be roughly balanced?

If data was inserted in random order, what would the expected tree depth be?

We can use a similar randomised analysis as quicksort

Consider a sequence of elements which we will insert into a BST

Q: What would be the best element to insert first? (i.e. the best value to have at the root)

Binary Search Tree Depth

How about just hoping that the tree will be roughly balanced?

If data was inserted in random order, what would the expected tree depth be?

We can use a similar randomised analysis as quicksort

Consider a sequence of elements which we will insert into a BST

Q: What would be the best element to insert first? (i.e. the best value to have at the root)

A: The median value – as for quicksort this would ensure that half the data will end up either side of this pivot value

Equally, we could define “good enough” choices of the next element to insert from the sequence and “bad” ones

As for quicksort, if we randomly select the next item to insert, we will get a mixture of good and bad choices throughout our tree

Binary Search Tree Depth

It turns out that inserting elements into a BST in random order will give expected tree depth:

$$h \approx 2.99 \log n = O(\log n)$$

So if we assume data arrives in random order we're ok – we'll get O(log n) performance

But worst case is still O(n)

Since we are only given the data to insert one element at a time, we can't randomly reorder it, as for quicksort

All we can say is:

"Dictionary operations on a BST will take O(log n) time with high probability if elements are inserted in random order"

Self-balancing Binary Search Trees

We need to be able to give stronger performance guarantees than this

Self-balancing binary search trees ensure that the BST always remains perfectly balanced

Q: Which operations will need modifying to maintain perfect balance?

Self-balancing Binary Search Trees

We need to be able to give stronger performance guarantees than this

Self-balancing binary search trees ensure that the BST always remains perfectly balanced

Q: Which operations will need modifying to maintain perfect balance?

A: Both insert and delete will modify the tree and potentially unbalance it, to maintain a balanced tree we will need to ***rebalance*** it

There are a number of strategies for maintaining a balanced tree, including: Red-Black trees, AVL trees, 2-3 trees, splay trees and B^+ trees

Mehlhorn covers (a,b)-trees and Red-Black trees in some detail

We will focus on AVL trees

AVL Trees

AVL trees maintain the following invariant:

The depths of a node's left and right subtrees may differ by at most 1

As a result, the depth of an AVL tree is limited to: $h = 1.44 \log_2 n$

The ***balance factor*** of a node is the depth of its right subtree minus the depth of its left subtree

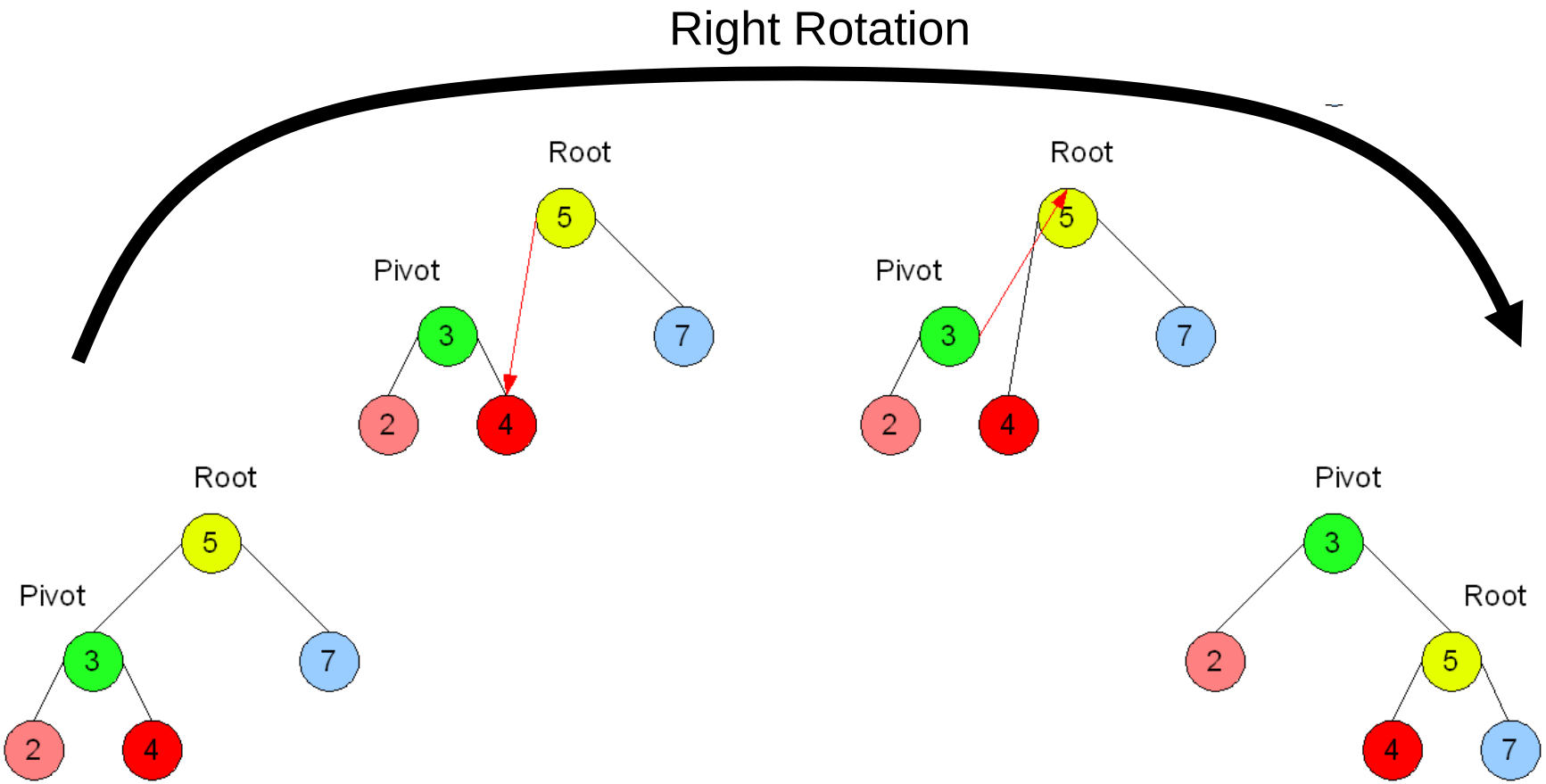
A node with a balance factor of -1, 0 or 1 is considered balanced

Any other balance factor requires rebalancing

Balance factors can be stored at each node or computed when needed

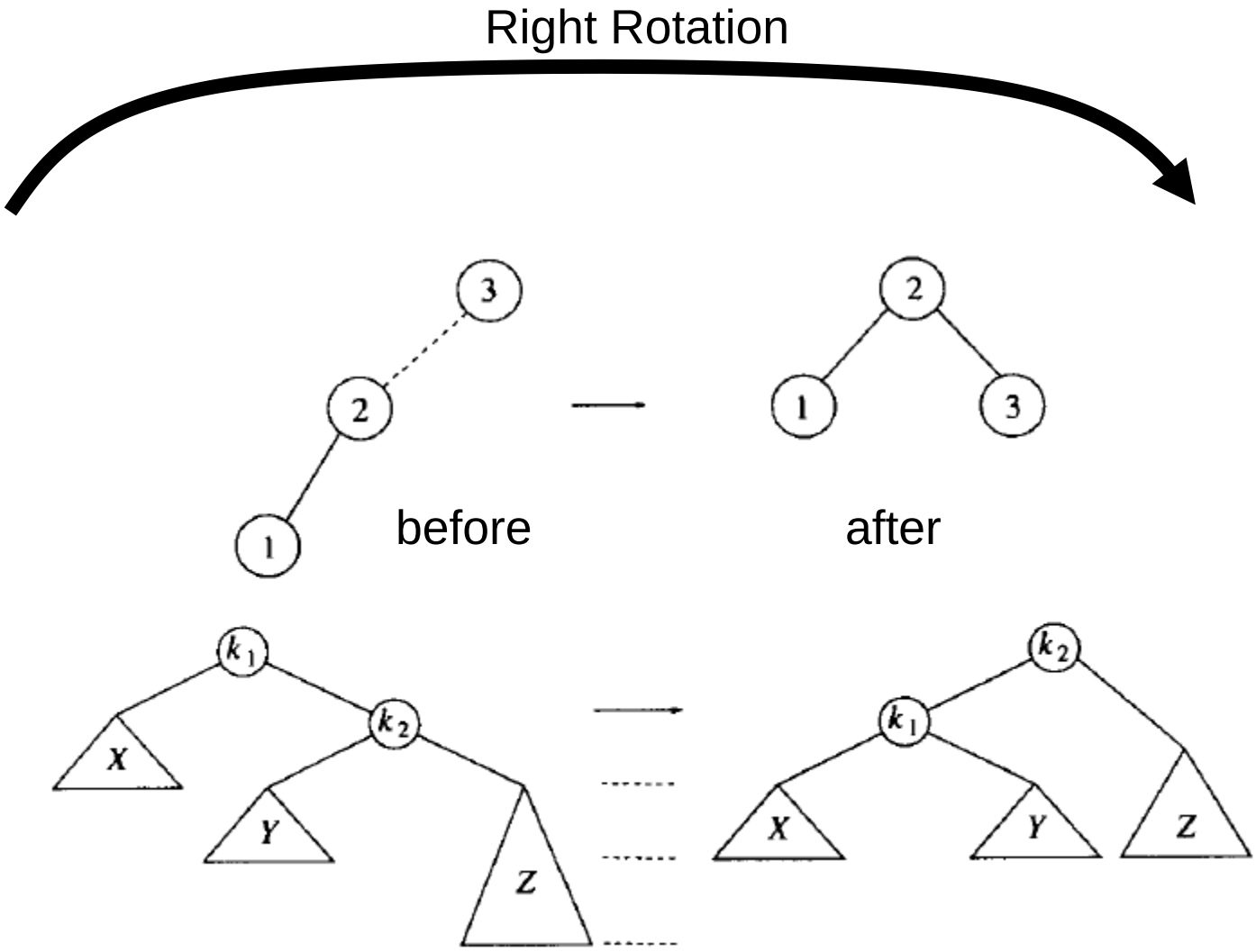
Basic idea: insert and delete as before, then check balance factors and use ***rotations*** to fix unbalanced subtrees

Tree Rotations



Root is the initial parent and **Pivot** is the child to take the root's place.

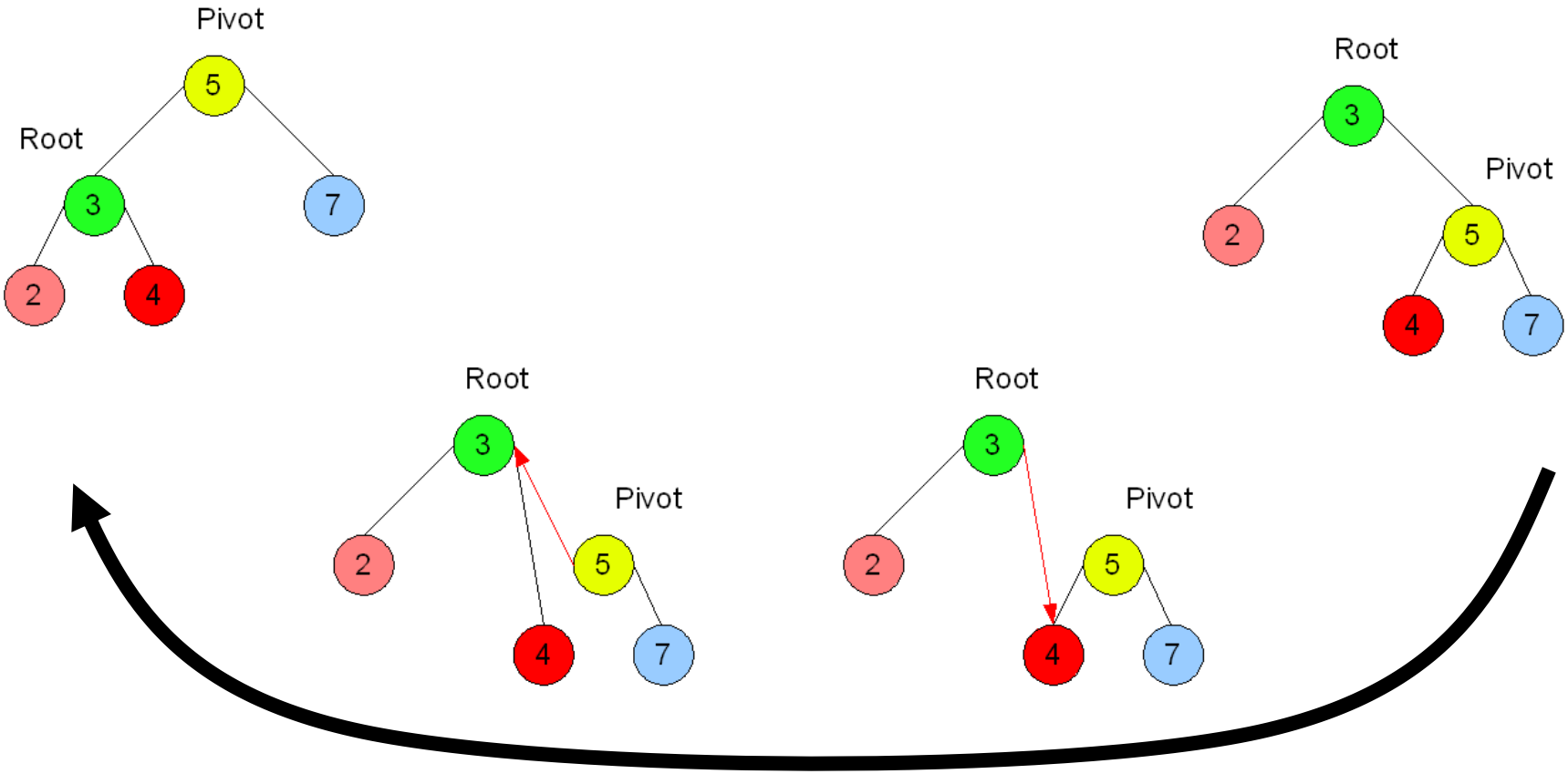
Tree Rotations



Tree Rotations

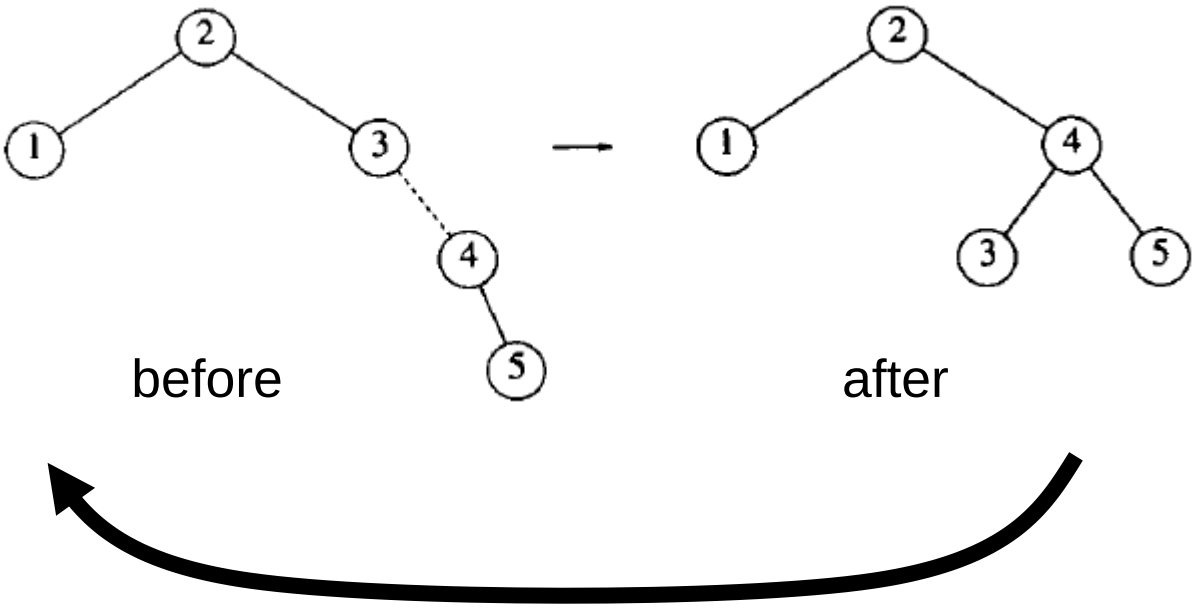
Root is the initial parent and **Pivot** is the child to take the root's place.

Final state **Initial state**



Left Rotation

Tree Rotations



Left Rotation

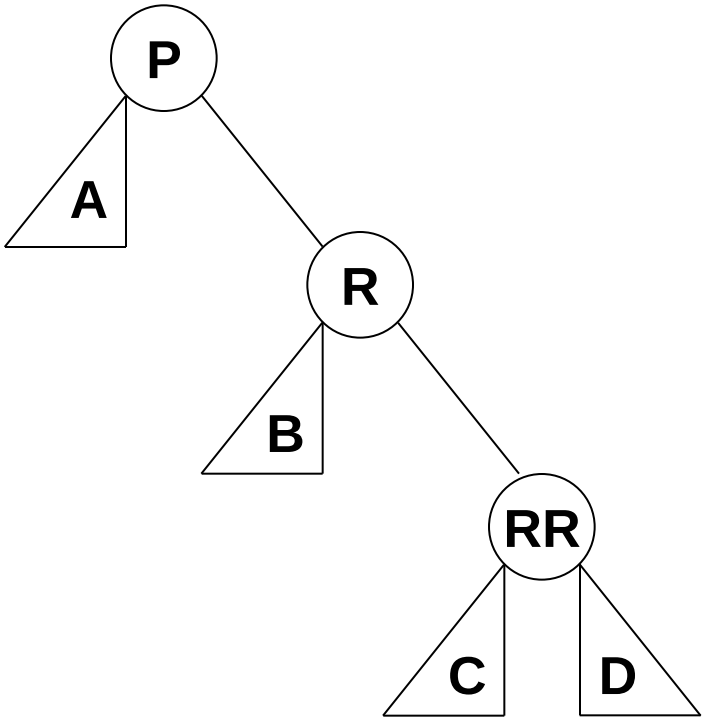
Inserting into an AVL Tree

- We insert as for an unbalanced BST
- We then retrace our steps back up the tree towards the root updating the balance factors as we go
- For each node as we move back up the tree we check the balance factor
- If it is -1, 0 or 1 then the subtree rooted at this node is in AVL form
- If it is 2 or -2 then the subtree rooted at this node is unbalanced
- There are 4 ways an unbalanced tree can occur (two for balance factor 2 and two for balance factor -2), the two pairs are symmetric so we only need to consider two cases

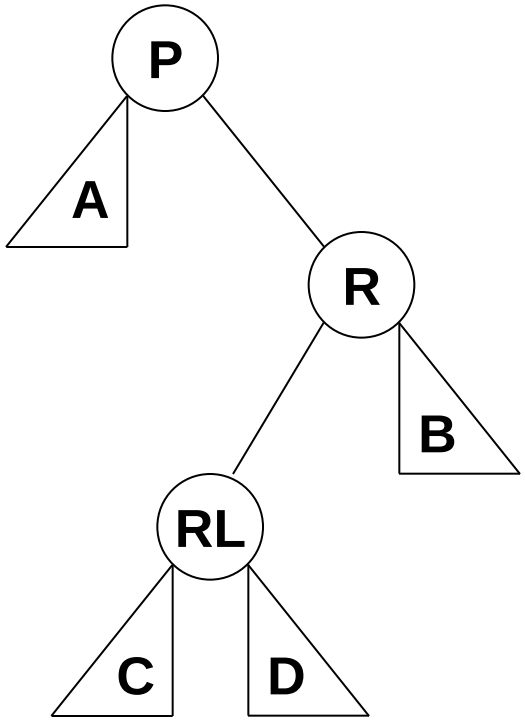
AVL Tree Rebalancing

Consider a subtree with root node P which has balance factor 2 (i.e. depth of right subtree outweighs left by 1)

Case 1 (Right right case):



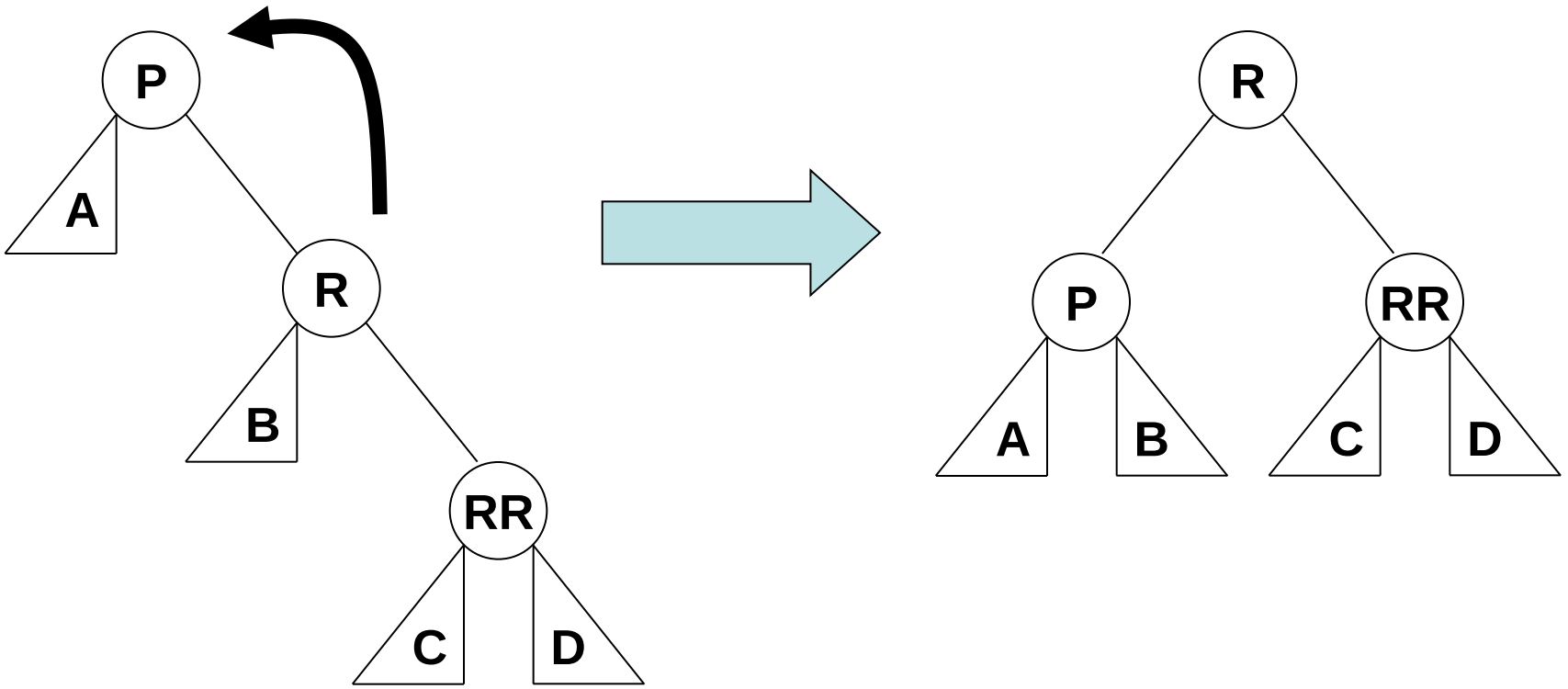
Case 2 (Right left case):



(A, B, C and D are perfectly balanced subtrees)

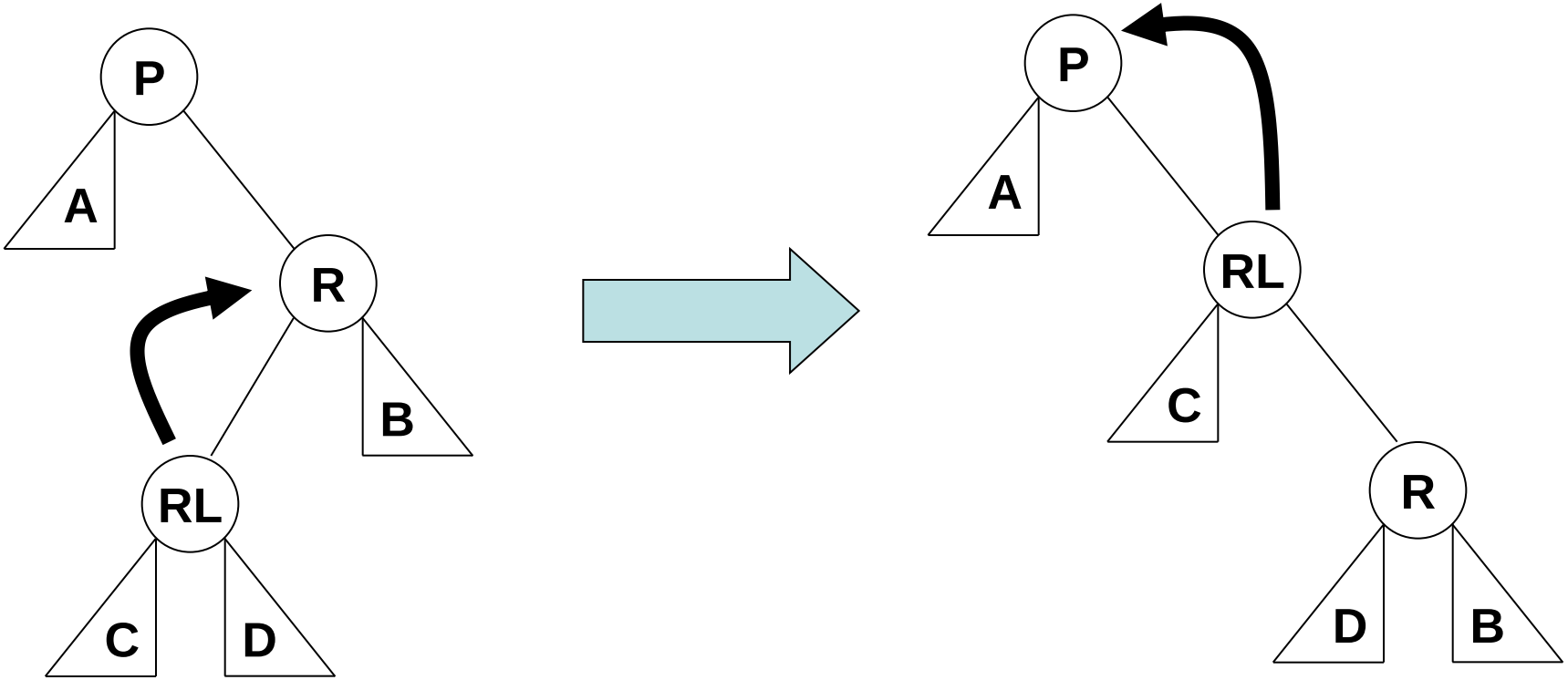
AVL Tree Rebalancing: Right Right Case

The right right case is fixed with a single left rotation about R:

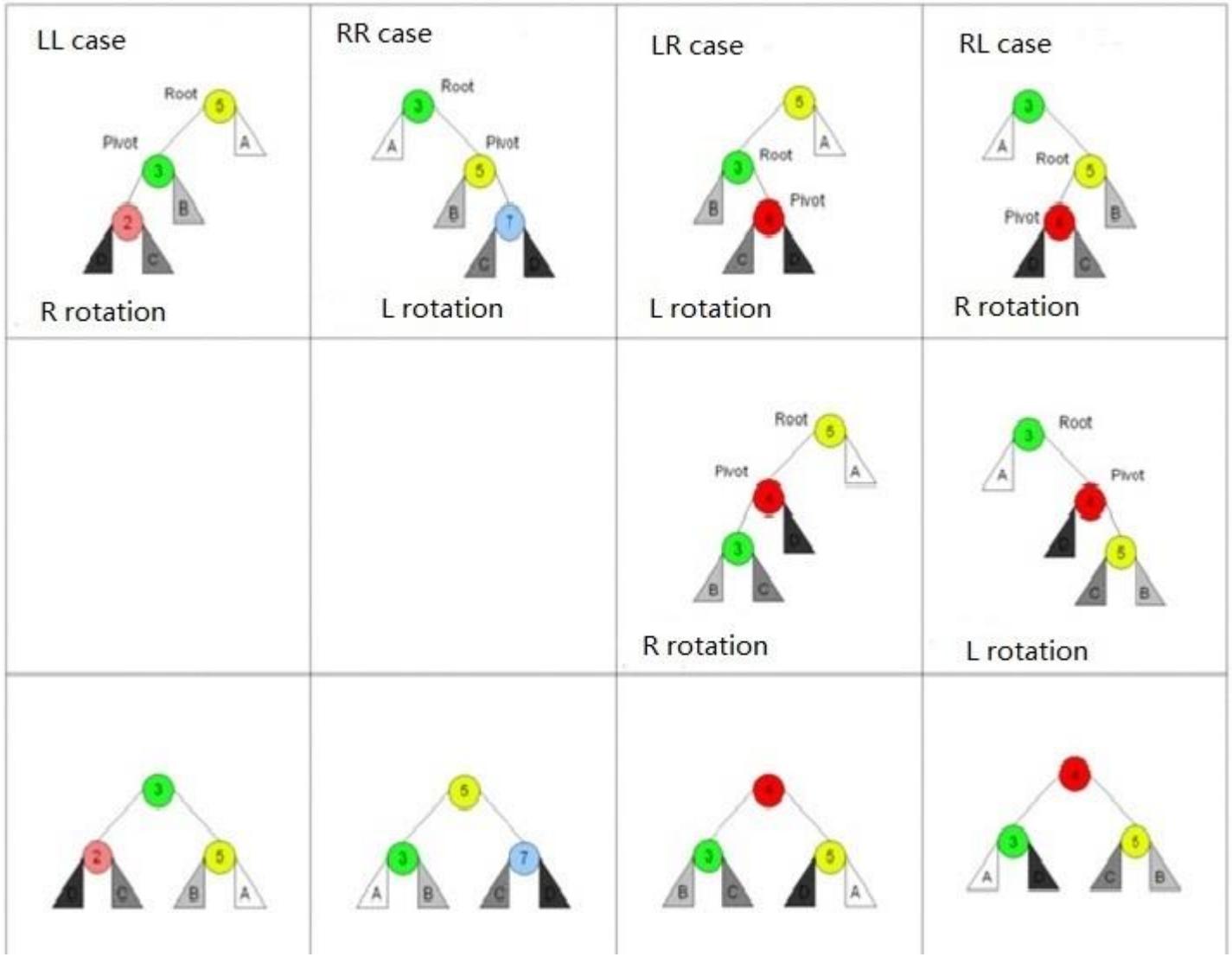


AVL Tree Rebalancing: Right left Case

The right left case is fixed with a right rotation about RL which converts the tree to a right right case, fixed as on previous slide by a further rotation:



Tree Rotations



AVL Trees

- We need to rebalance after an insert or delete operation

Q: What effect does this have on the complexity of these two operations?

AVL Trees

- We need to rebalance after an insert or delete operation

Q: What effect does this have on the complexity of these two operations?

A: In the worst case, we have to perform rotations at every node as we retrace our steps back to the root

- Rotations cost constant time = O(1)
- Distance from leaf to root is O(log n)
- This means rebalancing costs O(log n)
- Insertion/deletion costs O(log n) + O(log n) = O(log n)
- It makes no difference to the complexity!

Dictionaries as AVL Trees

All dictionary operations are O(log n) worst case complexity using an AVL tree:

Dictionary operation	Binary Search Tree
`Lookup(D,k)`	O(log n)
`Insert(D,v)`	O(log n)
`Delete(D,k)`	O(log n)
`IsPresent(D,k)`	O(log n)

Other Balanced Tree Structures

Balanced tree structures are defined by the invariant they maintain on the tree

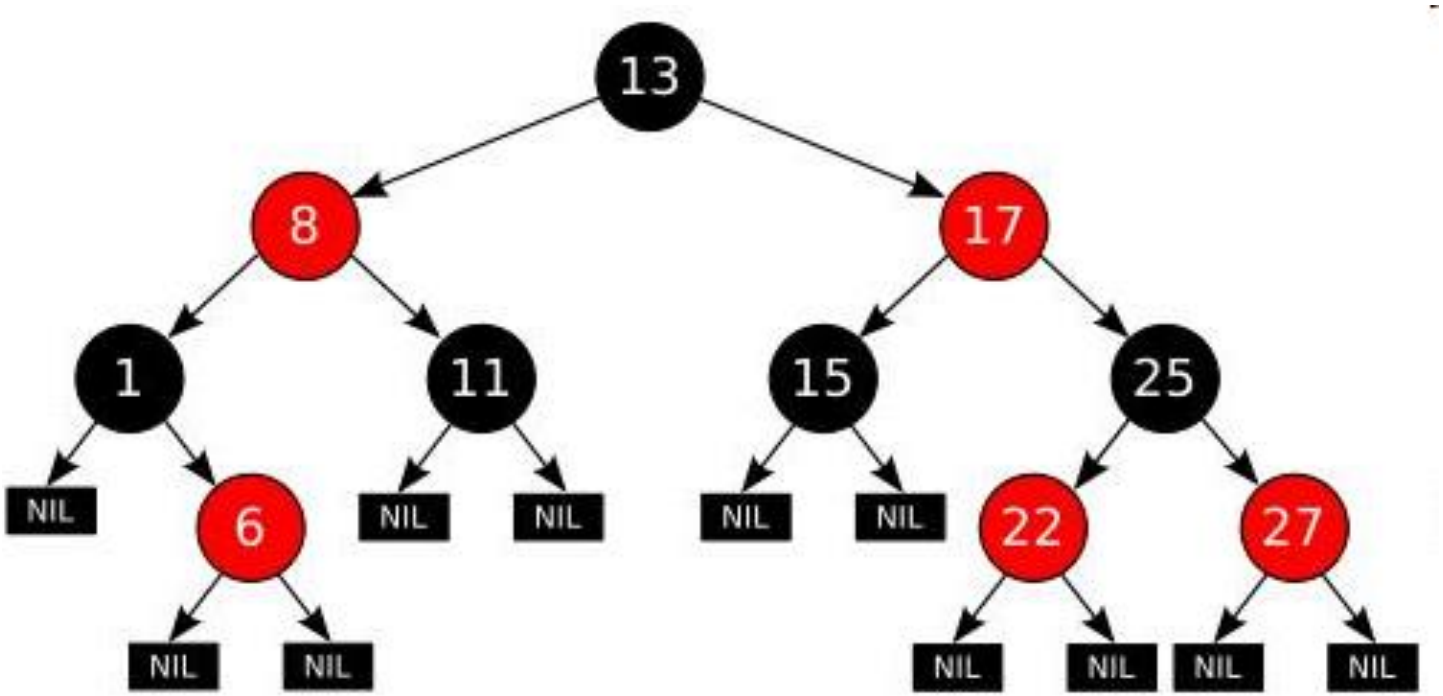
In a red-black tree, nodes are coloured either red or black

In addition:

1. The root is black
2. All leaves are black
3. Both children of every red node are black
4. Every path from a node to a descendent contains the same number of black nodes
5. Leaf nodes do not contain data

The effect of these requirements is that the longest path from the root to a leaf is no more than twice as long as the shortest path from root to leaf in the tree

Other Balanced Tree Structures



The depth of a red-black tree is limited to: $h = 2 \log_2 n$

Rebalancing is more complex than for an AVL tree

All dictionary operations can be executed in O(log n) time

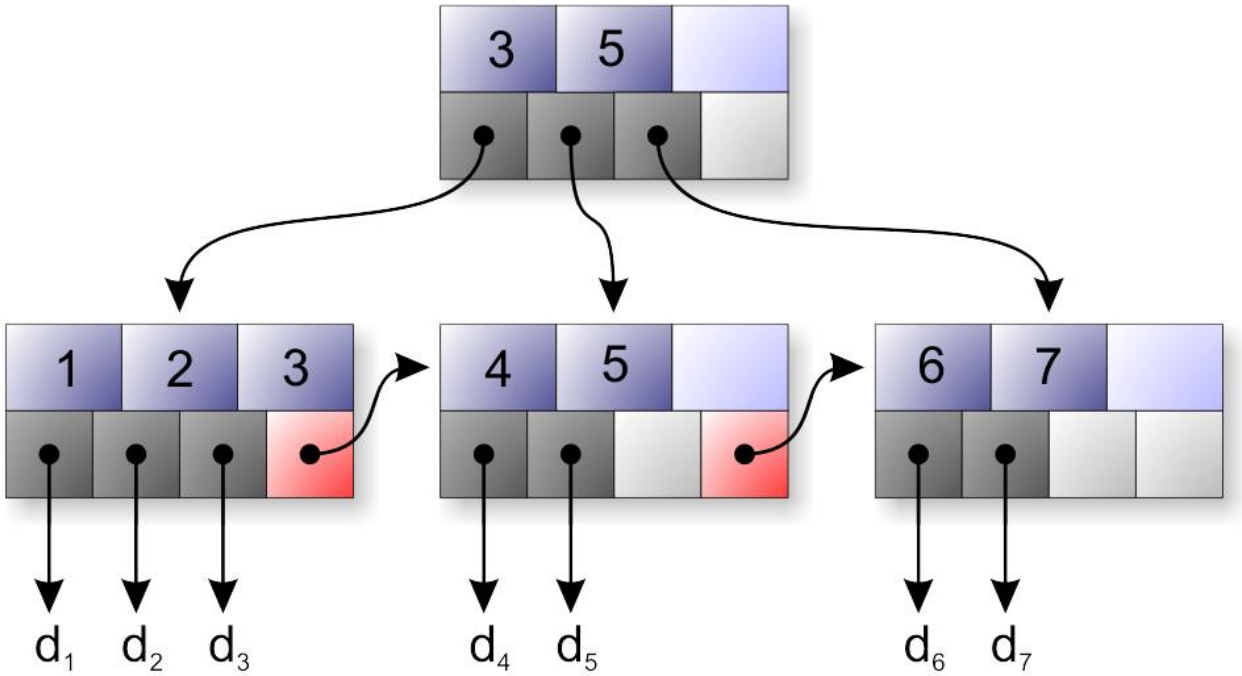
Other Balanced Tree Structures

B+-trees store all their data in leaves, only keys are stored in internal nodes

The invariant is that all leaves are at the same depth and that the root is either a leaf or has two or more children

In a tree of order m, internal nodes contain between m and 2m keys

B+- trees are highly efficient for block oriented storage



Sorting

Q: Can we use the data structures we've seen today to sort a sequence? What is the complexity?

Sorting

Q: Can we use the data structures we've seen today to sort a sequence? What is the complexity?

A: Insertions into an AVL tree are worst case O(log n).

We could take our n elements and insert all of them into an AVL tree at a total cost of O(n log n).

We can then read the items back out of the tree in order (in-order traversal) which costs linear time.

This would give us a sorted sequence in O(n)+O(n log n) = O(n log n) time

In practice, this is slower than quicksort or merge sort and requires O(n) auxiliary space

Conclusions

We should now know:

- What a Binary Search Tree is
- How to insert, retrieve, delete from a BST and traverse it in-order
- How to maintain a balanced tree using an AVL tree
- How dictionary operations can be efficiently implemented using an AVL tree

Next time:

- From keys to indexes efficiently: hash tables

Recommended reading for this lecture:

- Mehlhorn Chapter 7
- Cormen Chapters 12 and 13
- Skiena 3.4